INTERVIEW ANSWERS — PART 1 (TOPICS)

C# · OOPS · .NET Core · Entity Framework · SQL Server

SECTION 1: C# FUNDAMENTALS

Q1. What is the difference between Abstract class vs Interface?

Abstract Class: can have implementation (concrete methods), fields, constructors, access modifiers. Supports single inheritance only. Used for IS-A relationships among related classes. Interface: pure contract — method signatures (before C# 8), no fields, no constructors, all public by default. Multiple interfaces allowed. Used for IS-WHAT (capability) relationships.

Code:

```
// Abstract class
public abstract class Shape {
    public string Color = "Red";          // field allowed
    public abstract double Area();        // must override
    public virtual void Print() =>       // can override
        Console.WriteLine($"Color: {Color}");
}

// Interface
public interface IDrawable { void Draw(); }
public interface IResizable { void Resize(int factor); }

// Class inherits ONE base + MULTIPLE interfaces
public class Circle : Shape, IDrawable, IResizable {
    public double Radius { get; set; }
    public override double Area() => Math.PI * Radius * Radius;
    public void Draw() => Console.WriteLine("Drawing circle");
    public void Resize(int factor) => Radius *= factor;
}

// Rule of thumb:
// Abstract = shared code + state among related types
// Interface = contract / capability for unrelated types
```

Q2. What is Polymorphism? Explain with examples.

Polymorphism = "many forms". Two types: 1. Compile-time (static) — method overloading: same name, different parameters. 2. Runtime (dynamic) — method overriding via virtual/override: child decides behaviour at runtime.

Code:

```
// Compile-time polymorphism (overloading)
public class Logger {
    public void Log(string msg) => Console.WriteLine(msg);
    public void Log(string msg, int level) => Console.WriteLine($"[{level}] {msg}");
    public void Log(Exception ex) => Console.WriteLine(ex.Message);
}

// Runtime polymorphism (overriding)
public abstract class Animal {
    public virtual void Speak() => Console.WriteLine("...");
}
public class Dog : Animal {
    public override void Speak() => Console.WriteLine("Woof");
}
public class Cat : Animal {
    public override void Speak() => Console.WriteLine("Meow");
}

// Same base reference — different behaviour at runtime
```

```
Animal[] animals = { new Dog(), new Cat() };
foreach (var a in animals) a.Speak();   // Woof  Meow
```

Q3. Explain Encapsulation vs Abstraction.

Encapsulation: bundling data + methods, hiding internal state using access modifiers (private/protected). Controls how data is read/written. Abstraction: hiding implementation complexity, exposing only what is needed. "What to do" not "how to do it".

Code:

```
// Encapsulation — private field, public property
public class BankAccount {
    private decimal _balance;          // hidden
    public decimal Balance => _balance; // read-only view

    public void Deposit(decimal amount) {
        if (amount <= 0) throw new ArgumentException("Positive only");
        _balance += amount;
    }
    public bool Withdraw(decimal amount) {
        if (amount > _balance) return false;
        _balance -= amount;
        return true;
    }
}

// Abstraction — hide HOW payment works
public abstract class Payment {
    public abstract void Process(decimal amount); // what
}
public class CreditCard : Payment {
    public override void Process(decimal amount) {
        ValidateCard(); ChargeGateway(amount); SendReceipt(); // how (hidden)
    }
    private void ValidateCard() { /* ... */ }
    private void ChargeGateway(decimal a) { /* ... */ }
    private void SendReceipt() { /* ... */ }
}
```

Q4. What is the difference between var vs dynamic in C#?

var: type inferred at COMPILE time — still fully type-safe, zero runtime overhead. dynamic: type resolved at RUNTIME via reflection — bypasses compile-time checks, performance overhead.

Code:

```
var x = 42;          // int at compile time — intellisense works
var s = "hello";     // string — cannot do s = 10 later (compile error)

dynamic d = 42;
d = "now a string";  // OK at runtime — no compile error
d = new List<int>();  // also OK

// Use var:  always (clean, type-safe)
// Use dynamic: COM interop, JSON deserialization, ExpandoObject
dynamic expando = new System.Dynamic.ExpandoObject();
expando.Name = "Vamshi";   // property created at runtime
Console.WriteLine(expando.Name);
```

Q5. Explain the static keyword and static class in C#.

static member: belongs to the TYPE, not an instance. Shared across all instances. Accessed via class name. static class: cannot be instantiated, all members must be static. Used for utility / helper classes.

Code:

```
// Static member
```

```
public class Counter {
    public static int Total = 0;   // shared
    public int Id;
    public Counter() { Id = ++Total; }
}
var c1 = new Counter(); // Total = 1
var c2 = new Counter(); // Total = 2
Console.WriteLine(Counter.Total); // 2

// Static class — utility
public static class MathUtils {
    public static double Square(double n) => n * n;
    public static double CircleArea(double r) => Math.PI * r * r;
}
Console.WriteLine(MathUtils.Square(5));      // 25
Console.WriteLine(MathUtils.CircleArea(3));  // 28.27
```

Q6. Difference between Const vs Readonly.

const: compile-time constant, implicitly static, must be a literal, cannot be changed ever. readonly: runtime constant, can be set in constructor, supports complex types.

Code:

```
public class Config {
    public const int MaxRetries = 3;          // compile-time, literal only
    public const string AppName = "MyApp";

    public readonly DateTime StartedAt;       // runtime
    public readonly List<string> AllowedIPs;  // complex type OK

    public Config() {
        StartedAt  = DateTime.UtcNow;         // set in constructor
        AllowedIPs = new List<string> { "127.0.0.1" };
    }
}

// Config.MaxRetries = 5;      // ERROR — const
// cfg.StartedAt = DateTime.Now; // ERROR — readonly after ctor
```

Q7. Explain ref vs out keywords.

ref: pass by reference; caller MUST initialise first; method can read AND write. out: pass by reference for output; caller need NOT initialise; method MUST assign before return.

Code:

```
// ref — caller initialises, method reads and writes
void Double(ref int n) => n *= 2;
int num = 5;
Double(ref num);
Console.WriteLine(num);   // 10

// out — caller does not initialise; method must write
bool TryParse(string s, out int result) {
    if (int.TryParse(s, out result)) return true;
    result = 0; return false;
}
if (TryParse("42", out int val))
    Console.WriteLine(val);  // 42

// Swap using ref
void Swap(ref int a, ref int b) { int t = a; a = b; b = t; }
int x = 1, y = 2;
Swap(ref x, ref y);
Console.WriteLine($"{x},{y}");  // 2,1
```

Q8. What is Sealed class in C#?

A sealed class cannot be inherited. Prevents further derivation — used for security, performance, or design correctness. Also applicable to methods inside unsealed classes.

Code:

```
public sealed class Singleton {
    private static Singleton _i;
    private Singleton() { }
    public static Singleton Instance => _i ??= new Singleton();
}
// class SubSingleton : Singleton { }  // ERROR — sealed

// Sealed override — allows overriding but prevents further overrides
public class Base   { public virtual  void Run() => Console.WriteLine("Base"); }
public class Child  : Base { public sealed override void Run() =>
Console.WriteLine("Child"); }
public class GChild : Child { /* public override void Run() {} */  // ERROR — sealed }
```

Q9. What is method signature?

A method signature is its unique identity: METHOD NAME + PARAMETER TYPES (count, order, types). Return type is NOT part of the signature. Overloading is based on signature.

Code:

```
public class Calc {
    // Different signatures — legal overloads
    public int    Add(int a, int b)           => a + b;       // (int,int)
    public double Add(double a, double b)     => a + b;       // (double,double)
    public int    Add(int a, int b, int c)    => a + b + c;   // (int,int,int)

    // Same signature — ILLEGAL even if return type differs
    // public string Add(int a, int b) => ""; // COMPILE ERROR
}
```

Q10. What is the Order of constructor execution?

Static constructors run once when the type is first used: base static → derived static. Then instance constructors: base instance → derived instance (base runs before child).

Code:

```
class Animal {
    static Animal() => Console.WriteLine("1. Animal static ctor");
    public Animal()  => Console.WriteLine("3. Animal instance ctor");
}
class Dog : Animal {
    static Dog()  => Console.WriteLine("2. Dog static ctor");
    public Dog()  => Console.WriteLine("4. Dog instance ctor");
}
var d = new Dog();
// Output:
// 1. Animal static ctor
// 2. Dog static ctor
// 3. Animal instance ctor
// 4. Dog instance ctor
```

Q11. What is destructor and how many can be there in a class?

A destructor (finalizer ~ClassName) is called by GC before object memory is reclaimed. Only ONE allowed per class. Non-deterministic — you cannot predict when it runs. Prefer IDisposable pattern.

Code:

```
public class FileWrapper {
    ~FileWrapper() {   // only one allowed
        Console.WriteLine("Finalizer called by GC");
    }
```

```
}

// Preferred: IDisposable (deterministic cleanup)
public class DbConn : IDisposable {
    private SqlConnection _conn;
    public DbConn(string cs) => _conn = new SqlConnection(cs);
    public void Dispose() { _conn?.Dispose(); GC.SuppressFinalize(this); }
    ~DbConn() => Dispose();   // safety net
}

using var db = new DbConn("...");   // Dispose called automatically
```

Q12. Why do you need a destructor when GC is there?

GC manages MANAGED memory automatically. But unmanaged resources (file handles, native memory, OS handles, COM objects) are NOT tracked by GC. A destructor/finalizer provides a safety net to release unmanaged resources if Dispose() was never called.

Code:

```
// Without destructor — unmanaged handle leaks if user forgets Dispose()
public class NativeResource : IDisposable {
    private IntPtr _handle;   // unmanaged

    public NativeResource() => _handle = NativeLib.Open();

    public void Dispose() {
        if (_handle != IntPtr.Zero) {
            NativeLib.Close(_handle);
            _handle = IntPtr.Zero;
        }
        GC.SuppressFinalize(this);   // tell GC: finalizer not needed
    }

    ~NativeResource() => Dispose();  // safety net
}
```

Q13. What happens when constructor is private?

A private constructor prevents external instantiation. Common uses: Singleton pattern, static utility classes, factory methods that control object creation.

Code:

```
// Singleton — private ctor
public class AppConfig {
    private static readonly AppConfig _instance = new AppConfig();
    private AppConfig() { /* load config */ }
    public static AppConfig Instance => _instance;
    public string Theme { get; set; } = "Dark";
}
// var c = new AppConfig();     // ERROR
var cfg = AppConfig.Instance;   // OK

// Factory pattern
public class Temperature {
    private Temperature(double c) => Celsius = c;
    public double Celsius { get; }
    public static Temperature FromCelsius(double c) => new Temperature(c);
    public static Temperature FromFahrenheit(double f) => new Temperature((f - 32) /
1.8);
}
var t = Temperature.FromFahrenheit(100);   // 37.78 °C
```

Q14. If an Abstract class has a concrete method, can its child have the same method name and modify it?

Yes — using the override keyword (if virtual/abstract) or the new keyword (method hiding). Override is polymorphic; new hides without polymorphism.

Code:

```
public abstract class Vehicle {
    public abstract void Start();          // must override
    public virtual  void Stop()  => Console.WriteLine("Vehicle stop");  // can override
    public         void Refuel() => Console.WriteLine("Vehicle refuel"); // hide with new
}

public class Car : Vehicle {
    public override void Start()  => Console.WriteLine("Car start");   // polymorphic
    public override void Stop()   => Console.WriteLine("Car stop");    // polymorphic
    public new      void Refuel() => Console.WriteLine("Car refuel");  // hides — NOT
polymorphic
}

Vehicle v = new Car();
v.Start();    // Car start   (override — runtime dispatch)
v.Stop();     // Car stop    (override)
v.Refuel();   // Vehicle refuel (new — compile-time, Vehicle version called)
```

Q15. Abstract vs Static class

Abstract: blueprint for inheritance; can be instantiated only via subclass; supports polymorphism; can have instance & static members. Static: cannot be instantiated at all; all members must be static; cannot be inherited; used for utility functions.

Code:

```
// Abstract — designed for inheritance
public abstract class Report {
    public string Title { get; set; }
    public abstract void Generate();       // subclasses implement
    public void Save() => File.WriteAllText(Title + ".txt", "...");
}

// Static — utility, no instantiation
public static class StringHelper {
    public static string ToPascalCase(string s) =>
        string.Join("", s.Split(' ').Select(w =>
            char.ToUpper(w[0]) + w[1..].ToLower()));
}

// new Report()        — ERROR (abstract)
// new StringHelper()  — ERROR (static)
// Report r = new Invoice();    — OK (concrete child)
Console.WriteLine(StringHelper.ToPascalCase("hello world")); // HelloWorld
```

Q16. Why does abstract class have constructors?

Abstract classes can have constructors to initialise shared state (fields) that all subclasses need. The constructor is called via base() from the derived class. It cannot be called directly because the class cannot be instantiated directly.

Code:

```
public abstract class Animal {
    public string Name { get; }
    public int Age  { get; }

    protected Animal(string name, int age) {  // base ctor initialises common state
        Name = name;
        Age  = age;
    }
    public abstract void Speak();
}
```

```
public class Dog : Animal {
    public Dog(string name, int age) : base(name, age) { }   // calls base ctor
    public override void Speak() => Console.WriteLine($"{Name} says Woof!");
}

var d = new Dog("Rex", 3);
d.Speak();   // Rex says Woof!
```

Q17. When to use abstract vs interface — real-time example in C#

Use abstract class when related types share code/state (e.g. all reports share a header). Use interface when unrelated types share a capability (e.g. anything that can be serialised).

Code:

```
// Abstract class — shared code across related types
public abstract class BaseRepository<T> {
    protected readonly DbContext Db;
    public BaseRepository(DbContext db) => Db = db;   // shared state
    public T? GetById(int id) => Db.Set<T>().Find(id); // shared impl
    public abstract IEnumerable<T> GetAll();           // each must implement
}

// Interface — unrelated types share capability
public interface IExportable {
    byte[] ExportToPdf();
    byte[] ExportToCsv();
}

// Report uses both: inherits code from abstract, declares capability via interface
public class SalesReport : BaseRepository<Sale>, IExportable {
    public SalesReport(DbContext db) : base(db) { }
    public override IEnumerable<Sale> GetAll() => Db.Sales.ToList();
    public byte[] ExportToPdf() => /* generate PDF */ Array.Empty<byte>();
    public byte[] ExportToCsv() => /* generate CSV */ Array.Empty<byte>();
}
```

Q18. Implicit and Explicit Interface

Implicit: interface members are public on the class — callable without casting. Explicit: member is only accessible via the interface type — useful when a class implements two interfaces with the same method name, or to hide implementation details.

Code:

```
public interface IGreet { void Hello(); }
public interface IFarewell { void Hello(); }   // same name!

public class Greeter : IGreet, IFarewell {
    void IGreet.Hello()    => Console.WriteLine("Hi!");     // explicit
    void IFarewell.Hello() => Console.WriteLine("Bye!");    // explicit

    public void PublicHello() => Console.WriteLine("Hello!"); // implicit
}

var g = new Greeter();
g.PublicHello();               // Hello!
// g.Hello();                  // ERROR — not accessible directly
((IGreet)g).Hello();           // Hi!
((IFarewell)g).Hello();        // Bye!
```

Q19. What is default interface method and why do we need it?

C# 8+ allows interface methods to have a default implementation. Enables adding new methods to published interfaces without breaking existing implementors (backwards compatibility).

Code:

```
public interface ILogger {
    void LogInfo(string msg);
    void LogError(string msg);

    // Default implementation — added later; existing classes not forced to implement
    void LogWarning(string msg) =>
        Console.WriteLine($"[WARN] {msg}");
}

public class ConsoleLogger : ILogger {
    public void LogInfo(string msg)  => Console.WriteLine($"[INFO] {msg}");
    public void LogError(string msg) => Console.WriteLine($"[ERROR] {msg}");
    // LogWarning not overridden — uses default
}

ILogger log = new ConsoleLogger();
log.LogWarning("Disk 90% full");   // [WARN] Disk 90% full  (default)
```

Q20. Method overriding with example

Method overriding: child class provides its own implementation of a base class virtual/abstract method using the override keyword. The runtime version depends on the actual object type.

Code:

```
public class Discount {
    public virtual double Calculate(double price) => price * 0.05;  // 5%
}
public class FestivalDiscount : Discount {
    public override double Calculate(double price) => price * 0.20; // 20%
}
public class ClearanceDiscount : Discount {
    public override double Calculate(double price) => price * 0.50; // 50%
}

Discount d = new FestivalDiscount();
Console.WriteLine(d.Calculate(1000));   // 200 (runtime dispatch)
```

Q21. Method hiding with example

Method hiding (using new keyword): the child defines its own method with the same name, but it is NOT polymorphic. When accessed via base reference, the base version is called.

Code:

```
public class Base   { public        void Show() => Console.WriteLine("Base"); }
public class Child  : Base { public new void Show() => Console.WriteLine("Child"); }

Child  c = new Child();  c.Show();   // Child
Base   b = new Child();  b.Show();   // Base  ← hiding — base version called
// override would print Child here ^

// When to use: rarely — when you need to intentionally shadow a base method
// e.g., library class you cannot modify, or versioning scenarios
```

Q22. Is return type considered in method overloading?

NO. Return type is NOT part of the method signature for overload resolution. Two methods that differ only in return type are a compile error. Overloading is based on name + parameter list only.

Code:

```
public class Parser {
    public int    Parse(string s) => int.Parse(s);     // sig: Parse(string)
    public double Parse(double d) => d;                // sig: Parse(double) — different
param
    // public long Parse(string s) => 0;              // ERROR — same sig as line 1
```

```
}

// These ARE different (parameter types differ)
public int    GetId(int code) => code;
public string GetId(string name) => name;   // OK — different param type
```

Q23. Why and when to use method hiding? Real-time scenario

Use method hiding (new) when: (a) you use a library class you cannot modify, (b) the base method is not virtual, and (c) you want a different behaviour for your type without breaking polymorphism of the base.

Code:

```
// Third-party library — cannot change
public class ThirdPartyLogger {
    public void Log(string msg) => Console.WriteLine($"[OLD] {msg}");
}

// Our class — we want enhanced Log without virtual base
public class EnhancedLogger : ThirdPartyLogger {
    public new void Log(string msg) =>
        Console.WriteLine($"[{DateTime.Now:HH:mm:ss}] {msg}");
}

EnhancedLogger el = new EnhancedLogger();
el.Log("Test");              // [14:30:00] Test   (our version)

ThirdPartyLogger tpl = el;
tpl.Log("Test");             // [OLD] Test         (base version — hiding, not
overriding)
```

Q24. How to extend a sealed class?

You CANNOT inherit a sealed class. Instead, use: (a) extension methods to add methods without inheritance, or (b) composition — wrap the sealed class inside a new class.

Code:

```
// Sealed — cannot inherit
public sealed class EmailSender {
    public void Send(string to, string body) => Console.WriteLine($"To: {to}");
}

// Option 1: Extension methods (add new methods, no subclassing)
public static class EmailSenderExt {
    public static void SendWelcome(this EmailSender sender, string email) =>
        sender.Send(email, "Welcome to our platform!");
}
var es = new EmailSender();
es.SendWelcome("vamshi@qa.com");   // works!

// Option 2: Composition
public class EnhancedEmailSender {
    private readonly EmailSender _inner = new EmailSender();
    public void Send(string to, string body) => _inner.Send(to, body);
    public void SendBulk(IEnumerable<string> tos, string body) {
        foreach (var to in tos) _inner.Send(to, body);
    }
}
```

Q25. What is extension method? Write an example

Extension methods add new methods to existing types without modifying them. Defined in a static class with the this keyword on the first parameter. They appear as instance methods on the type.

Code:

```
public static class IntExtensions {
```

```
    public static bool IsEven(this int n) => n % 2 == 0;
    public static int  Squared(this int n) => n * n;
    public static IEnumerable<int> Range(this int from, int to) =>
        Enumerable.Range(from, to - from + 1);
}

Console.WriteLine(4.IsEven());  // True
Console.WriteLine(5.Squared()); // 25
foreach (var n in 1.Range(5)) Console.Write(n + " "); // 1 2 3 4 5
```

Q26. Purpose of the this keyword in extension methods

The this keyword on the first parameter marks the method as an extension method and tells the compiler which type it extends. Without it the method is just a regular static method. It does NOT mean "instance of this class".

Code:

```
public static class StringExt {
    // 'this string s' — extends the string type
    public static string Truncate(this string s, int maxLen) =>
        s.Length <= maxLen ? s : s[..maxLen] + "...";
}

string title = "Hello World from C# Interview";
Console.WriteLine(title.Truncate(11));  // Hello World...

// Internally the compiler converts:
// title.Truncate(11) → StringExt.Truncate(title, 11)
```

Q27. Write an extension method example

A practical extension on List<T> to chunk a list into pages.

Code:

```
public static class ListExtensions {
    public static IEnumerable<List<T>> Paginate<T>(this List<T> source, int pageSize) {
        for (int i = 0; i < source.Count; i += pageSize)
            yield return source.GetRange(i, Math.Min(pageSize, source.Count - i));
    }
}

var employees = Enumerable.Range(1, 10).ToList();
foreach (var page in employees.Paginate(3)) {
    Console.WriteLine(string.Join(", ", page));
}
// 1, 2, 3
// 4, 5, 6
// 7, 8, 9
// 10
```

Q28. Extension method: given DateTime, return Morning / Afternoon / Evening

Code:

```
public static class DateTimeExtensions {
    public static string GetTimeOfDay(this DateTime dt) => dt.Hour switch {
        >= 5  and < 12 => "Morning",
        >= 12 and < 17 => "Afternoon",
        >= 17 and < 21 => "Evening",
        _              => "Night"
    };
}

Console.WriteLine(DateTime.Now.GetTimeOfDay());
Console.WriteLine(new DateTime(2025,1,1,9,0,0).GetTimeOfDay());   // Morning
Console.WriteLine(new DateTime(2025,1,1,14,0,0).GetTimeOfDay());  // Afternoon
Console.WriteLine(new DateTime(2025,1,1,19,0,0).GetTimeOfDay());  // Evening
```

SECTION 2: OOPS & DESIGN CONCEPTS

Q1. Difference between List, Hashtable & Dictionary

List<T>: ordered, index-based, allows duplicates. Use for sequences. Hashtable: non-generic key-value, no type safety, thread-safe (read), legacy. Dictionary<K,V>: generic key-value, type-safe, fast O(1) lookup, not thread-safe by default.

Code:

```
// List<T>
var list = new List<int> { 3, 1, 2, 1 };   // duplicates OK
list.Add(4);  Console.WriteLine(list[0]);   // 3 (index-based)

// Dictionary<K,V>
var dict = new Dictionary<string, int> {
    ["Alice"] = 90, ["Bob"] = 85
};
dict["Charlie"] = 92;
Console.WriteLine(dict["Alice"]);    // 90
dict.TryGetValue("Dave", out int score); // safe lookup

// Hashtable (legacy — avoid in new code)
var ht = new Hashtable { ["key1"] = "val1" };
Console.WriteLine(ht["key1"]);  // val1 (no type-safety, needs cast)
```

Q2. What is Array & ArrayList?

Array: fixed size, strongly typed, fast. ArrayList: dynamic size, non-generic (stores object), requires boxing/unboxing, legacy. Prefer List<T> over ArrayList.

Code:

```
// Array — fixed size, typed
int[] nums = new int[3] { 1, 2, 3 };
nums[0] = 10;
// nums[5] = 1; // IndexOutOfRangeException

// ArrayList — dynamic, non-generic (avoid)
var al = new ArrayList { 1, "hello", 3.14 };  // mixed types
al.Add(true);
int n = (int)al[0];   // cast required

// Preferred: List<T> — dynamic + type-safe
var lst = new List<string> { "A", "B" };
lst.Add("C");
Console.WriteLine(lst.Count);  // 3
```

Q3. What is Delegate and where have you used it?

A delegate is a type-safe function pointer — it holds a reference to a method matching a specific signature. Used for callbacks, events, LINQ, and strategy patterns.

Code:

```
// Delegate declaration
delegate int Operation(int a, int b);

// Usage
Operation add = (a, b) => a + b;
Operation mul = (a, b) => a * b;
Console.WriteLine(add(3, 4));  // 7
Console.WriteLine(mul(3, 4));  // 12

// Built-in delegates
```

```
Action<string> log = msg => Console.WriteLine(msg);
Func<int,int,int> sum = (a, b) => a + b;
Predicate<int> isEven = n => n % 2 == 0;

log("Hello");          // Hello
Console.WriteLine(sum(2, 3));    // 5
Console.WriteLine(isEven(4));    // True

// Real use: event handler
button.Click += (s, e) => Console.WriteLine("Clicked");

// Real use: strategy (pass sorting logic)
List<int> nums = new List<int> { 3,1,4,1,5 };
nums.Sort((a, b) => b - a);  // descending via delegate
```

Q4. What is an extension method?

(Same as C# Q25 — see above) Extension methods add methods to existing types without inheritance. Defined as static methods with this on the first parameter.

Q5. Difference between Ref & Out?

ref: caller must initialise before passing; method can read and write. out: caller need not initialise; method must write before returning.

Code:

```
void AddTax(ref decimal price) => price *= 1.18m;   // reads existing value
decimal p = 100;
AddTax(ref p); Console.WriteLine(p);   // 118

bool TryGetUser(int id, out string name) {
    name = id == 1 ? "Vamshi" : null;
    return name != null;
}
if (TryGetUser(1, out string n)) Console.WriteLine(n);  // Vamshi
```

Q6. What is the ASP.Net Page life cycle?

Request → Init → Load → Validate → Event → Render → Unload. (Web Forms). In MVC/Core: Request → Routing → Controller selection → Action execution → Result → Response.

Code:

```
// ASP.NET Web Forms Page Life Cycle (in order):
// 1. PreInit       — master page, theme selection
// 2. Init          — controls initialised
// 3. InitComplete  — initialisation done
// 4. PreLoad
// 5. Load          — Page_Load() here
// 6. Control Events — Button_Click etc.
// 7. LoadComplete
// 8. PreRender     — last chance to modify controls
// 9. SaveViewState
// 10. Render       — HTML generated
// 11. Unload       — cleanup

// ASP.NET Core MVC Request Pipeline:
// Request → Middleware chain → Routing → Authorization → Model binding
//         → Action filters → Action method → Result filters → View/JSON → Response
```

Q7. How can we maintain cache / cookie in ASP.NET page?

Cache: IMemoryCache or IDistributedCache (Redis) in .NET Core. Cookie: HttpContext.Response.Cookies.

Code:

```
// Cache (in-memory)
// Register: services.AddMemoryCache();
public class ProductController : Controller {
    private readonly IMemoryCache _cache;
    public ProductController(IMemoryCache cache) => _cache = cache;

    public IActionResult Index() {
        if (!_cache.TryGetValue("products", out List<string> products)) {
            products = new List<string> { "A", "B", "C" };  // DB call
            _cache.Set("products", products, TimeSpan.FromMinutes(5));
        }
        return Ok(products);
    }
}

// Cookie — read/write
Response.Cookies.Append("UserTheme", "dark", new CookieOptions {
    Expires = DateTimeOffset.UtcNow.AddDays(30),
    HttpOnly = true, Secure = true, SameSite = SameSiteMode.Strict
});
string theme = Request.Cookies["UserTheme"];
```

Q8. How data can be passed from one controller to another from MVC?

Options: TempData, Session, Query string, Route values, or redirect with route parameters. TempData is most common for single-redirect data.

Code:

```
// Option 1: TempData (survives one redirect)
public IActionResult CreateOrder() {
    TempData["OrderId"] = 42;
    return RedirectToAction("Confirmation", "Order");
}
// In OrderController:
public IActionResult Confirmation() {
    int orderId = (int)TempData["OrderId"];
    return View(orderId);
}

// Option 2: Query string
return RedirectToAction("Details", "Product", new { id = 5, tab = "reviews" });
// URL: /Product/Details?id=5&tab=reviews

// Option 3: Session
HttpContext.Session.SetString("Cart", JsonSerializer.Serialize(cart));
// In another controller:
var cart = JsonSerializer.Deserialize<Cart>(HttpContext.Session.GetString("Cart"));
```

Q9. Difference between ViewBag & ViewData?

Both carry data from controller to view for the SAME request. ViewData: dictionary (string key), requires casting. ViewBag: dynamic wrapper around ViewData — no casting but no IntelliSense.

Code:

```
// Controller
public IActionResult Index() {
    ViewData["Title"] = "Home";          // string key, object value
    ViewBag.Message  = "Welcome!";       // dynamic property

    ViewData["Count"] = 42;
    return View();
}

// View (.cshtml)
// @ViewData["Title"]           → Home
```

```
// @ViewBag.Message             → Welcome!
// @((int)ViewData["Count"])    → 42 (cast needed for ViewData)
// @ViewBag.Count               → 42 (no cast for ViewBag)

// Same underlying dictionary — ViewBag.X == ViewData["X"]
```

Q10. Difference between Response.Redirect() & Server.Transfer()

Response.Redirect(): tells the browser to navigate to a new URL (two HTTP round trips, URL changes). Server.Transfer(): server-side only (one request, URL stays the same in browser, faster, only for same server).

Code:

```
// Response.Redirect (WebForms / MVC)
// Browser receives 302 → makes NEW request to /Home/Index
Response.Redirect("/Home/Index");

// Server.Transfer (WebForms only)
// Server executes /OtherPage.aspx — browser URL unchanged
Server.Transfer("OtherPage.aspx");

// In ASP.NET Core MVC equivalent:
return Redirect("/Home/Index");              // 302 redirect
return RedirectToAction("Index", "Home");    // 302 redirect

// No Server.Transfer equivalent in Core — use RedirectToAction
// For permanent redirect:
return RedirectPermanent("/new-url");  // 301
```

Q11. Difference between Middleware & Filters in .NET Core?

Middleware: works at HTTP pipeline level — runs for EVERY request, before MVC routing. Handles CORS, auth, compression, logging globally. Filters: work inside MVC pipeline — apply to controllers/actions, access action context, ModelState, etc.

Code:

```
// Middleware — runs for everything
app.Use(async (ctx, next) => {
    Console.WriteLine("Before — any request");
    await next();
    Console.WriteLine("After — any response");
});

// Action Filter — runs only for MVC actions
public class LogActionFilter : IActionFilter {
    public void OnActionExecuting(ActionExecutingContext ctx) =>
        Console.WriteLine($"Before action: {ctx.ActionDescriptor.DisplayName}");
    public void OnActionExecuted(ActionExecutedContext ctx) =>
        Console.WriteLine("After action");
}

// Apply filter on controller
[ServiceFilter(typeof(LogActionFilter))]
public class OrderController : Controller { /* ... */ }

// Key difference:
// Middleware: no access to controllers/models, runs before routing
// Filters: MVC-aware, access ModelState, ActionDescriptor, Result
```

Q12. How the middleware architecture works end to end

Request enters middleware pipeline → each middleware decides to call next() or short-circuit → action executes → response travels back through each middleware in reverse.

Code:

```
// Pipeline (order matters)
```

```
app.UseExceptionHandler("/Error");   // outermost — catch all
app.UseHttpsRedirection();
app.UseStaticFiles();                // short-circuits for static files
app.UseRouting();
app.UseAuthentication();             // who are you?
app.UseAuthorization();              // what can you do?
app.MapControllers();

// Flow visualised:
// → ExceptionHandler → Https → Static → Routing → Auth → AuthZ → Controller
// ← ExceptionHandler ← Https ← Static ← Routing ← Auth ← AuthZ ← Response
```

Q13. Why are we creating middleware? What's the main use?

Middleware handles cross-cutting concerns that apply to ALL or MOST requests: global exception handling, request logging, authentication checks, response compression, CORS, rate limiting, custom headers.

Q14. Have you used/created custom middlewares?

Yes. Example: request-ID injection middleware that stamps every request with a unique ID for tracing.

Code:

```
public class RequestIdMiddleware {
    private readonly RequestDelegate _next;
    public RequestIdMiddleware(RequestDelegate next) => _next = next;

    public async Task InvokeAsync(HttpContext ctx) {
        var id = Guid.NewGuid().ToString("N")[..8];
        ctx.Response.Headers["X-Request-Id"] = id;
        ctx.Items["RequestId"] = id;
        await _next(ctx);
    }
}
// Register
app.UseMiddleware<RequestIdMiddleware>();
```

Q15. What is short-circuit in middleware?

Short-circuit means a middleware returns a response WITHOUT calling next() — the remaining pipeline is skipped. Used for auth failures, IP blocks, static files.

Code:

```
app.Use(async (ctx, next) => {
    if (ctx.Request.Headers["X-Api-Key"] != "secret-key") {
        ctx.Response.StatusCode = 401;
        await ctx.Response.WriteAsync("Unauthorised");
        return;          // ← SHORT CIRCUIT — next() NOT called
    }
    await next();        // continue pipeline
});

// app.Run() always short-circuits (terminal middleware)
app.Run(async ctx => {
    await ctx.Response.WriteAsync("Terminal — no next()");
});
```

Q16. What is the advantage of .NET Core compared to .NET Framework?

Cross-platform (Windows/Linux/Mac), open-source, high performance (Kestrel), side-by-side versioning, smaller footprint, built-in DI, Docker/cloud-native ready.

Q17. How do we define the scope of an object in .NET Core?

Register in DI container with AddTransient, AddScoped, or AddSingleton in Program.cs.

Code:

```
builder.Services.AddTransient<IEmailService, EmailService>();  // new every injection
builder.Services.AddScoped<IOrderRepo, OrderRepo>();           // new per HTTP request
builder.Services.AddSingleton<ICacheService, CacheService>();  // one for app lifetime
```

Q18. How many ways can we define dependency injection?

Three ways: Constructor injection (recommended), Property injection, Method injection.

Code:

```
// 1. Constructor injection (PREFERRED)
public class OrderService {
    private readonly IRepo _repo;
    public OrderService(IRepo repo) => _repo = repo;  // injected via ctor
}

// 2. Property injection (via [FromServices] or manual)
public class ReportController : Controller {
    [FromServices] public ILogger<ReportController> Logger { get; set; }
}

// 3. Method injection (rare, for action methods)
public IActionResult Get([FromServices] IProductService svc) =>
    Ok(svc.GetAll());
```

Q19. How are we defining/managing security for your APIs?

JWT bearer tokens for authentication, role/policy-based authorisation, HTTPS enforced, CORS configured, input validation, rate limiting, secrets in Azure Key Vault.

Code:

```
// JWT setup
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(opt => {
        opt.TokenValidationParameters = new TokenValidationParameters {
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(secret)),
            ValidateIssuer = true,   ValidIssuer   = "myapp",
            ValidateAudience = true, ValidAudience = "users",
            ValidateLifetime = true
        };
    });

// Authorisation policy
builder.Services.AddAuthorization(opt =>
    opt.AddPolicy("AdminOnly", p => p.RequireRole("Admin")));

// Controller
[Authorize(Policy = "AdminOnly")]
[HttpDelete("{id}")]
public IActionResult Delete(int id) => Ok();
```

Q20. How are we generating the JWT token and after that how to validate it?

Code:

```
// Generate
public string GenerateToken(User user) {
    var key   = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_secret));
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
    var token = new JwtSecurityToken(
        issuer:   "myapp",
        audience: "users",
        claims:   new[] {
            new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
            new Claim(ClaimTypes.Role, user.Role)
```

```
        },
        expires:           DateTime.UtcNow.AddHours(1),
        signingCredentials: creds);
    return new JwtSecurityTokenHandler().WriteToken(token);
}

// Validate — done automatically by AddJwtBearer middleware
// Manual validate:
var handler = new JwtSecurityTokenHandler();
handler.ValidateToken(tokenString, validationParams, out SecurityToken validated);
var jwt   = (JwtSecurityToken)validated;
var userId = jwt.Claims.First(c => c.Type == ClaimTypes.NameIdentifier).Value;
```

Q21. Difference between REST & API?

API (Application Programming Interface): any interface that lets two systems communicate (REST, SOAP, GraphQL, RPC). REST: a specific architectural STYLE for APIs over HTTP using stateless request-response with resources identified by URLs.

Q22. What are the protocols used in REST APIs?

HTTP/HTTPS (standard). REST is protocol-agnostic by definition but in practice almost always uses HTTP. Methods: GET, POST, PUT, PATCH, DELETE.

INTERVIEW ANSWERS — PART 2

.NET Core · Entity Framework · SQL Server · Design Patterns · SOLID

SECTION 3: .NET CORE & ASP.NET (120 Questions)

Q1. What is CLR (Common Language Runtime)?

CLR is the runtime engine of .NET. It compiles IL (Intermediate Language) to native machine code via JIT, manages memory (GC), enforces type safety, handles exceptions, and provides security sandboxing.

Q2. Difference between Managed vs Unmanaged code.

Managed: executed by CLR — automatic GC, type-safe (C#, VB.NET). Unmanaged: executed directly by OS — manual memory (C++, Assembly). Interop via P/Invoke.

Q3. What are Generics in C#?

Generics allow type-safe, reusable classes/methods with a type parameter. Avoids boxing/unboxing overhead. E.g. List<T>, Dictionary<K,V>.

Q4. What are Extension Methods in C#?

Static methods with this on first parameter that appear as instance methods on the extended type. Used extensively in LINQ.

Q5. What are Delegates in C#?

Type-safe function pointers. Hold references to methods matching a signature. Built-ins: Action, Func, Predicate.

Q6. Explain the 'using' statement in C#.

Ensures IDisposable objects are disposed even if an exception occurs. Equivalent to try-finally with Dispose().

Q7. Difference between IEnumerable vs IQueryable.

IEnumerable: LINQ to Objects — filters in-memory. IQueryable: translates queries to SQL — filters on server. Use IQueryable for DB queries.

Q8. Explain Garbage Collection in C#.

GC automatically reclaims unreachable managed memory. Three generations: Gen 0 (short-lived), Gen 1, Gen 2 (long-lived). Call GC.Collect() only in exceptional cases.

Q9. Difference between async/await and Task/Thread.

Thread: OS-level, heavy. Task: lightweight work item, pool-backed. async/await: non-blocking syntax over Task — keeps thread free while waiting.

Q10. What is Parallel Programming in C#?

Running work on multiple CPU cores simultaneously. Parallel.For, Parallel.ForEach, PLINQ, Task.WhenAll.

Q11. Explain Middleware in ASP.NET Core.

Request-processing components chained in a pipeline. Each can handle the request, delegate to next(), or short-circuit.

Q12. How to create Custom Middleware in ASP.NET Core?

Class with RequestDelegate constructor parameter and InvokeAsync(HttpContext) method. Register with app.UseMiddleware<T>().

Q13. Difference between Dependency Injection lifetimes.

Transient: new every injection. Scoped: new per HTTP request. Singleton: one instance for app lifetime.

Q14. Explain Authentication vs Authorization.

Authentication: WHO are you? (identity — JWT, cookies). Authorization: WHAT can you do? (permissions — roles, policies).

Q15. What is JWT (JSON Web Token)?

Three base64url-encoded parts separated by dots: Header (alg+typ), Payload (claims), Signature (HMAC/RSA). Stateless — server verifies without session store.

Q16. Explain REST API principles.

6 constraints: Client-Server, Stateless, Cacheable, Uniform Interface, Layered System, Code on Demand (optional).

Q17. Difference between PUT vs PATCH.

PUT: replace entire resource (idempotent). PATCH: partial update (idempotent in practice). Both are idempotent.

Q18. How to handle Global Exception Handling in .NET Core?

Middleware (ExceptionHandlingMiddleware) or UseExceptionHandler(). Catches all unhandled exceptions, returns consistent error response.

Q19. How to secure APIs from common vulnerabilities?

JWT/OAuth auth, HTTPS, input validation, parameterised queries (SQL injection), CORS policy, rate limiting, secrets in Key Vault, [Authorize] attributes.

Q20. What is Kestrel web server?

Built-in, cross-platform, high-performance HTTP server in ASP.NET Core. Serves requests directly or behind a reverse proxy (IIS/Nginx).

Q21. Why Kestrel when we have IIS?

Kestrel is cross-platform and faster. IIS/Nginx act as reverse proxies: SSL termination, load balancing, static files, security hardening.

Q22. Explain concept of reverse proxy?

A server that receives client requests and forwards them to backend servers. Client talks to proxy, not the backend directly. Provides SSL, caching, load balancing.

Q23. What are cookies?

Small key-value data stored in browser, sent with every request to the server. Used for session tracking, auth tokens, preferences.

Q24. What is the need for session management?

HTTP is stateless — sessions store user-specific state (cart, auth) between requests without sending it every time.

Q25. Explain 'HTTP is a stateless protocol'?

Each HTTP request is independent — server does not remember previous requests. State must be maintained explicitly (session, cookies, JWT).

Q26. Are sessions enabled by default?

No. Must register: services.AddSession() + app.UseSession() in ASP.NET Core.

Q27. How to enable sessions in MVC Core?

services.AddDistributedMemoryCache(); services.AddSession(); app.UseSession();

Q28. Are session variables shared (global) between users?

No. Each user has their own session identified by a session cookie. Sessions are private per user.

Q29. Do session variables use cookies?

Yes. A session ID cookie (e.g. .AspNetCore.Session) is sent to the browser; the actual data is stored server-side.

Q30. Explain idle timeout in sessions?

Session expires after a period of inactivity. Configurable: options.IdleTimeout = TimeSpan.FromMinutes(20);

Q31. What does a Context mean in HTTP?

HttpContext: per-request object containing Request, Response, User, Session, Items, Cookies. Shared across middleware/controller for one request.

Q32. When should we use ViewData?

For passing small pieces of data from controller to view in the same request. Prefer ViewModel for type safety.

Q33. How to pass data from controller to view?

ViewData/ViewBag (weakly typed), ViewModel (strongly typed, preferred), TempData (survives redirect).

Q34. In same request can ViewData persist across actions?

No. ViewData only lives for one request. For redirect use TempData.

Q35. ViewBag vs ViewData?

Both carry data controller→view. ViewData: Dictionary<string,object>, needs cast. ViewBag: dynamic wrapper around ViewData, no cast.

Q36. How does ViewBag work internally?

ViewBag is a dynamic property of ControllerBase backed by ViewData. ViewBag.X == ViewData["X"].

Q37. Explain viewModels?

Strongly typed classes designed specifically for a view, containing exactly the data the view needs. Best practice.

Q38. ViewBag vs ViewModel — best practice?

ViewModel always. Type-safe, IntelliSense, model binding, validation. ViewBag only for trivial one-off values.

Q39. Explain TempData?

Short-lived storage that survives ONE redirect. Backed by session or cookie. Used for flash messages, redirect data.

Q40. Can TempData persist across action redirects?

Yes — that is its purpose. Data from one redirect is available in the next action, then deleted.

Q41. How is TempData different from ViewData?

TempData: survives redirects, deleted after read. ViewData: same request only, no redirect.

Q42. If TempData is read is it available for next request?

No — it is marked for deletion after being read. Use TempData.Keep("key") to retain it.

Q43. How to persist TempData?

TempData.Keep() — keeps all. TempData.Keep("key") — keeps specific key. TempData.Peek("key") — reads without marking for deletion.

Q44. What does Keep do in TempData?

Marks TempData entries to not be deleted after reading — effectively keeps them for the next request too.

Q45. Explain Peek in TempData?

Reads TempData value WITHOUT marking it for deletion. Safe read — value still available next time.

Q46. How is TempData different from session variables?

TempData: auto-deleted after one redirect (1-2 requests). Session: persists until timeout or explicit clear.

Q47. If I restart the server does TempData, session stay?

In-memory: NO — lost. With distributed cache (Redis) or DB: YES — survives restarts.

Q48. Is TempData private to a user?

Yes. Backed by per-user session/cookie — cannot see another user's TempData.

Q49. ViewData vs ViewBag vs TempData vs Session

ViewData/ViewBag: same request, controller→view. TempData: one redirect. Session: across multiple requests until timeout.

Q50. What is WebAPI?

ASP.NET Web API is a framework for building HTTP services (REST) consumed by clients (browser, mobile, other services).

Q51. What is the advantage of WebAPI?

HTTP native, stateless, supports multiple formats (JSON/XML), language-agnostic, works with any client, testable.

Q52. Explain REST and Architectural constraints of REST?

1.Client-Server 2.Stateless 3.Cacheable 4.Uniform Interface 5.Layered System 6.Code on Demand.

Q53. Can we use UDP/TCPIP protocol with Web API?

Standard ASP.NET Web API uses HTTP/HTTPS (TCP-based). For UDP you would use raw sockets or SignalR WebSockets, not REST.

Q54. How WebAPI different from MVC controller?

MVC returns Views (HTML). WebAPI returns data (JSON/XML). In ASP.NET Core they are unified — ControllerBase for API, Controller for MVC.

Q55. What is content negotiations in Web API?

Server and client agree on response format via Accept header. Client sends Accept: application/json or Accept: application/xml.

Q56. WebAPI vs WCF?

WebAPI: lightweight, HTTP only, REST, JSON/XML, easy. WCF: heavy, supports multiple protocols (TCP, HTTP, MSMQ), SOAP, contract-first.

Q57. WCF REST vs WebAPI REST?

Both can do REST but WebAPI is simpler, faster to develop, better JSON support, community/Microsoft preferred.

Q58. How to return HTTP status codes?

return Ok(data) 200 | return Created() 201 | return NoContent() 204 | return BadRequest() 400 | return NotFound() 404 | return StatusCode(500)

Q59. For error which status code is returned?

4xx: client errors (400 Bad Request, 401 Unauthorised, 403 Forbidden, 404 Not Found). 5xx: server errors (500 Internal Server Error, 503 Service Unavailable).

Q60. How did you secure your web API?

JWT bearer auth, [Authorize] attribute, HTTPS, input validation, CORS policy, rate limiting, Key Vault for secrets.

Q61. How do current JS frameworks work with WebAPI?

Angular/React call API via HttpClient/fetch, parse JSON response, display data. They send JWT in Authorization header.

Q62. .NET Core vs .NET Framework differences

.NET Core: cross-platform, open-source, high performance, Docker/cloud-native, side-by-side versions. .NET Framework: Windows-only, mature, legacy support.

Q63. Difference between .NET core and traditional .NET

.NET Core (now ".NET 5+") is the modern, cross-platform successor. .NET Framework is Windows-only legacy.

Q64. Can we implement startup.cs file in .NET 6.0?

Yes but it is optional. .NET 6+ uses minimal hosting (Program.cs with top-level statements). You can still add a Startup class manually.

Q65. Dot net core 5.0 vs 6.0

.NET 6: LTS, minimal API, C# 10, hot reload, Maui, WebApplication builder (no Startup.cs required). .NET 5: STS, unified platform.

Q66. ConfigureServices vs Configure method?

ConfigureServices: register DI services. Configure: define middleware pipeline. In .NET 6 both are in Program.cs via builder.Services and app.Use*.

Q67. Explain the importance of Startup.cs file in ASP.NET Core?

Two methods: ConfigureServices (register DI) + Configure (build middleware pipeline). Central config file (legacy approach, replaced by Program.cs in .NET 6+).

Q68. Where do we store configuration in ASP.NET Core?

appsettings.json (+ appsettings.{env}.json), environment variables, Azure Key Vault, User Secrets (dev), command-line args.

Q69. How can we read configuration file in ASP.NET Core?

IConfiguration injected via DI: _config["MySection:Key"] or IOptions<T> for strongly-typed config.

Q70. Explain the various way of doing DI in MVC?

Constructor injection (recommended), Property injection ([FromServices]), Method injection (action parameter with [FromServices]).

Q71. Explain Razor pages?

Page-based model where each page has a .cshtml view and a .cshtml.cs PageModel (code-behind). Simpler than MVC for page-centric apps.

Q72. How can we pass model data to the view?

return View(myModel); — model bound to @model MyType in view. Also ViewData/ViewBag for loose data.

Q73. What is a strongly typed view?

View with @model TypeName at top — compiler checks property names, IntelliSense works, reduces runtime errors.

Q74. Explain the concept of viewModel?

A POCO class built specifically for a view containing exactly what it needs (not the domain entity directly). Decouples domain from presentation.

Q75. What is CORS policy?

Cross-Origin Resource Sharing — browser security mechanism. Server explicitly allows cross-domain AJAX calls via Access-Control-Allow-Origin headers.

Q76. Why CORS no issue in Postman but in browser?

Postman does not enforce CORS — it is a browser security policy. The browser checks server headers and blocks if CORS not configured.

Q77. What is Restful API?

An API following REST principles: stateless, resources identified by URLs, HTTP verbs (GET/POST/PUT/DELETE), returns JSON/XML.

Q78. Basic REST API principles.

Stateless, Client-Server, Cacheable, Uniform Interface (resources + verbs + representations), Layered.

Q79. SOAP vs REST

SOAP: strict XML, WSDL contract, protocol-agnostic, enterprise, verbose. REST: lightweight, JSON, HTTP, no strict contract, easier to consume.

Q80. What is JSON?

JavaScript Object Notation — lightweight, human-readable data format. Key-value pairs and arrays. Language-agnostic. Default for REST APIs.

Q81. JSON vs XML

JSON: compact, readable, native JS, faster parsing. XML: verbose, supports namespaces/schemas, SOAP required it.

Q82. POST vs PUT (idempotent).

POST: create new resource, NOT idempotent (calling twice creates two records). PUT: replace entire resource, idempotent (same result each call).

Q83. POST vs PUT vs Patch.

POST=create(non-idempotent), PUT=full replace(idempotent), PATCH=partial update(idempotent).

Q84. If We can modify resource via POST why we need PUT?

POST is NOT idempotent — calling twice creates duplicates. PUT is idempotent — safe to retry. REST convention clarity.

Q85. Can GET call have request body?

Technically HTTP allows it but strongly discouraged. GET should be safe and idempotent. Use query params instead.

Q86. Can we add resource using GET call?

You should NOT. GET must be safe (no side effects). Use POST to create.

Q87. What is Content negotiations in Web API?

Server returns format based on Accept header. Clients requesting JSON get JSON; clients requesting XML get XML.

Q88. Different Status Code and their usages.

200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorised, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable, 500 Internal Server Error.

Q89. Difference between 404 Not Found and 500 Internal Server Error.

404: client error — resource does not exist. 500: server error — server crashed/bug. Client cannot fix 500.

Q90. If REST API is slow what will you do, where will you hunt, how to optimize?

Check: DB queries (N+1, missing indexes), API profiling (Application Insights), caching, async I/O, payload size, connection pooling, CDN.

Q91. What is API Versioning and its type? How to implement it

Types: URL versioning (/api/v1/), Query string (?ver=1), Header versioning (X-Api-Version), Media type. Use Microsoft.AspNetCore.Mvc.Versioning NuGet.

Q92. Return types in .NET Core?

IActionResult, ActionResult<T>, specific types (string, int), Task<IActionResult> for async.

Q93. Can we have multiple return types in .NET Core?

Yes — use IActionResult or ActionResult<T>. One action can return Ok(data), NotFound(), BadRequest() etc.

Q94. Different return types in Web API.

IActionResult (most flexible), ActionResult<T> (typed + flexible), void/Task (204), specific type (automatic 200).

Q95. Difference between Web API and Minimal API.

Minimal API: less boilerplate, lambda-based routes, no controller classes (ideal for microservices). Web API: controller-based, full MVC pipeline, filters, model binding.

Q96. How to implement sorting and paging in .NET Core?

Query params: ?page=1&pageSize=10&sortBy=name. LINQ: .Skip((page-1)*size).Take(size).OrderBy(x => x.Name).

Q97. How to implement Authentication/Authorization in .NET Core?

AddAuthentication + AddJwtBearer in services. app.UseAuthentication() + app.UseAuthorization() in pipeline. [Authorize] on controllers.

Q98. What is JWT? State its parts. How it is implemented?

Header.Payload.Signature. Issued on login, sent in Authorization: Bearer header, validated by middleware on each request.

Q99. How did you implement JWT logout?

Client deletes the token. Server-side: add token to a blacklist (Redis/DB) and check on each request. Shorten expiry + use refresh tokens.

Q100. When should we use OpenId, OAuth vs Custom JWT?

Use OpenID Connect / OAuth when: social login, third-party SSO, Azure AD / Google. Custom JWT: simple internal auth, full control.

Q101. How to save sensitive data in database?

Passwords: bcrypt/Argon2 hash (never encrypt). PII: AES-256 encryption. Keys in Azure Key Vault. Connection strings in environment variables.

Q102. What are some common encryption techniques?

Symmetric: AES-256. Asymmetric: RSA, ECC. Hashing: SHA-256, bcrypt (passwords). TLS for transport.

Q103. What data encryption techniques you have used in project?

AES-256 for data at rest, TLS/HTTPS for transit, bcrypt for passwords, Azure Key Vault for key management.

Q104. Explain API Authorization (JWT / OAuth).

JWT: self-contained token with claims. OAuth: delegation framework (user grants app access to their resource). OpenID Connect: identity layer on OAuth.

Q105. How do you revoke tokens with 30-minute expiry?

Maintain a token blacklist (Redis set). On logout add token JTI claim to blacklist. Middleware checks blacklist before processing.

Q106. How do you invalidate tokens from UI?

Client deletes local token (localStorage/cookie). Server blacklists JTI. For instant revocation: short expiry + refresh token rotation.

Q107. If a user logs out from one browser, what happens in another browser session?

Without blacklist: both sessions still valid (JWT is stateless). With blacklist: logout blacklists JTI — all sessions using that token revoked.

Q108. Encryption vs Hashing – differences and use cases.

Encryption: reversible (AES) — data that needs to be read back. Hashing: one-way (bcrypt) — passwords (never need to decrypt).

Q109. How do you block specific IPs and where is it configured?

Middleware (IP filtering), API Gateway (Azure APIM policies), WAF (Web Application Firewall), or Nginx/IIS IP restrictions.

Q110. How would you design cloud-scale security for billions of users?

Azure AD B2C for identity, OAuth 2.0, short-lived JWTs + refresh tokens, Azure Front Door WAF, DDOS protection, Key Vault, zero-trust architecture.

Q111. What is API gateway?

Single entry point for all clients. Handles: routing, auth, rate limiting, SSL termination, logging, load balancing. Azure APIM, Kong, AWS API Gateway.

Q112. What is Rate limiter?

Controls request rate per client/IP to prevent abuse/DDOS. ASP.NET Core 7+ has built-in: services.AddRateLimiter().

Q113. Suppose there is a requirement to upload a file with lakhs of records in database from MVC.

Stream the file, use SqlBulkCopy or EF Core bulk extensions (EFCore.BulkExtensions), batch in chunks of 1000, use async, background job (Hangfire/Azure Function).

Q114. How to do it so transaction will be faster

SqlBulkCopy: inserts 100k rows in seconds. Disable constraints during load, re-enable after. Use explicit transaction. Parallel batches.

Q115. Can you explain the MVC lifecycle and flow of request from browser to server and back?

Browser → DNS → Server → Middleware pipeline → Routing → Auth → Model Binding → Action → ActionResult → View Engine → HTML → Browser.

Q116. What is cache management? How can it be implemented in React?

React: React Query, SWR, Redux cache, localStorage, sessionStorage. Cache API calls, invalidate on mutation.

Q117. How do we manage caching in ASP.NET MVC? What about ASP.NET Core Web API?

MVC: [OutputCache] attribute, Response.Cache. Core API: IMemoryCache, IDistributedCache (Redis), [ResponseCache] attribute, ETag.

Code Examples for Key .NET Core Questions

Q9b. async/await vs Task/Thread — Code

Code:

```
// Thread — heavy, manual management
var thread = new Thread(() => Console.WriteLine("Thread running"));
thread.Start();

// Task — lightweight, pool-backed
var task = Task.Run(() => Console.WriteLine("Task running"));
await task;

// async/await — non-blocking, readable
public async Task<List<Product>> GetProductsAsync() {
    // Thread is FREE while waiting for DB
    var products = await _db.Products.ToListAsync();
    return products;
}

// Parallel — CPU-bound work
int[] numbers = Enumerable.Range(1, 1_000_000).ToArray();
int sum = 0;
Parallel.ForEach(numbers, n => Interlocked.Add(ref sum, n));
```

Q13b. DI Lifetimes — Code

Code:

```
// Registration
builder.Services.AddTransient<IOperationTransient, Operation>();
builder.Services.AddScoped<IOperationScoped, Operation>();
builder.Services.AddSingleton<IOperationSingleton, Operation>();

// Demonstration
public class HomeController : Controller {
    public HomeController(
        IOperationTransient t1, IOperationTransient t2, // different instances
        IOperationScoped   s1, IOperationScoped   s2,  // same per request
        IOperationSingleton sg1, IOperationSingleton sg2) // always same
    {
        Console.WriteLine(t1.Id == t2.Id);   // False (Transient)
        Console.WriteLine(s1.Id == s2.Id);   // True  (Scoped)
        Console.WriteLine(sg1.Id == sg2.Id); // True  (Singleton)
    }
}
```

Q15b. JWT Generation & Validation — Code

Code:

```
// --- GENERATE ---
var key   = new SymmetricSecurityKey(Encoding.UTF8.GetBytes("super-secret-key-min-
32chars!!!!"));
var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

var token = new JwtSecurityToken(
    issuer:            "myapp.com",
    audience:          "myapp-users",
    claims:            new[] {
        new Claim(ClaimTypes.NameIdentifier, "42"),
```

```
        new Claim(ClaimTypes.Name,           "Vamshi"),
        new Claim(ClaimTypes.Role,           "Admin")
    },
    expires:           DateTime.UtcNow.AddHours(1),
    signingCredentials: creds
);
string jwt = new JwtSecurityTokenHandler().WriteToken(token);

// --- VALIDATE (auto via middleware) ---
// appsettings.json → Jwt:Key, Jwt:Issuer, Jwt:Audience
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(opt => {
        opt.TokenValidationParameters = new() {
            ValidateIssuerSigningKey = true,
            IssuerSigningKey         = key,
            ValidateIssuer           = true, ValidIssuer   = "myapp.com",
            ValidateAudience         = true, ValidAudience = "myapp-users",
            ValidateLifetime         = true
        };
    });

// --- USE ---
[Authorize]
[HttpGet("profile")]
public IActionResult GetProfile() {
    var userId = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
    return Ok(new { userId });
}
```

Q94b. API Versioning — Code

Code:

```
// Install: Microsoft.AspNetCore.Mvc.Versioning
builder.Services.AddApiVersioning(opt => {
    opt.ReportApiVersions      = true;
    opt.AssumeDefaultVersionWhenUnspecified = true;
    opt.DefaultApiVersion      = new ApiVersion(1, 0);
    opt.ApiVersionReader       = new UrlSegmentApiVersionReader();
});

// URL versioning: /api/v1/products  /api/v2/products
[ApiController]
[ApiVersion("1.0")]
[Route("api/v{version:apiVersion}/products")]
public class ProductsV1Controller : ControllerBase {
    [HttpGet] public IActionResult Get() => Ok("v1 products");
}

[ApiController]
[ApiVersion("2.0")]
[Route("api/v{version:apiVersion}/products")]
public class ProductsV2Controller : ControllerBase {
    [HttpGet] public IActionResult Get() => Ok("v2 products — enhanced");
}
```

Q99b. Sorting & Paging — Code

Code:

```
public async Task<IActionResult> GetEmployees(
    [FromQuery] int page = 1,
    [FromQuery] int pageSize = 10,
    [FromQuery] string sortBy = "Name",
    [FromQuery] string order = "asc") {

    var query = _db.Employees.AsQueryable();
```

```
    // Dynamic sort
    query = sortBy switch {
        "Salary" => order == "asc" ? query.OrderBy(e => e.Salary)
                                   : query.OrderByDescending(e => e.Salary),
        _        => order == "asc" ? query.OrderBy(e => e.Name)
                                   : query.OrderByDescending(e => e.Name)
    };

    int total = await query.CountAsync();
    var items  = await query.Skip((page - 1) * pageSize).Take(pageSize).ToListAsync();

    return Ok(new { total, page, pageSize, items });
}
```

SECTION 4: ENTITY FRAMEWORK & ORM (25 Questions)

Q1. What is ORM?

Object-Relational Mapper: maps database tables to C# classes. Eliminates boilerplate SQL. Examples: EF Core, Dapper, NHibernate.

Q2. Have you used any ORM in your project? Which one?

EF Core (Code First, LINQ queries, migrations) and Dapper (raw SQL with mapping) for performance-critical queries.

Q3. What is Dapper?

Micro-ORM: executes raw SQL and maps results to C# objects. Faster than EF Core for read-heavy scenarios. No migrations/change tracking.

Q4. What is EF Core?

Full ORM by Microsoft: LINQ-to-SQL, change tracking, migrations, lazy/eager loading, Code First/Database First.

Q5. EF Core vs EF Framework?

EF Core: cross-platform, .NET Core/5+, faster, more features (filtered includes, TPT, etc.). EF Framework: Windows/.NET Framework only, legacy.

Q6. What are navigation properties in EF Core?

Properties that represent relationships between entities. EF Core uses them for JOINs. E.g. public List<Order> Orders in Customer class.

Q7. How did you implement EF Core with relational database?

DbContext with DbSet<T> properties. OnModelCreating for Fluent API config. Migrations for schema management. Connection string from appsettings.json.

Q8. Which approach did you follow in your project?

Code First — define models in C#, generate DB from migrations. Gives full control over schema evolution.

Q9. Code first vs Database first?

Code First: create models → generate DB (new projects, clean slate). Database First: existing DB → scaffold models (legacy databases).

Q10. Commands used for code first

dotnet ef migrations add InitialCreate | dotnet ef database update | dotnet ef migrations remove | dotnet ef dbcontext scaffold

Q11. What are fluent APIs?

Method chaining in OnModelCreating to configure entity mappings without data annotations. More powerful and explicit than attributes.

Q12. How do you call a Stored Procedure from EF core?

_db.Database.ExecuteSqlRawAsync("EXEC sp_Name @p0", value) or _db.Set<T>().FromSqlRaw("EXEC sp_Name {0}", value).ToListAsync().

Q13. What is lazy loading?

Navigation property data loaded ON DEMAND — only when accessed. Requires UseLazyLoadingProxies(). Can cause N+1 query problem.

Q14. Eager loading in EF Core?

Load related data up-front using .Include() and .ThenInclude(). Single DB query with JOIN. Preferred for known relationships.

Q15. Tracking and Non Tracking in EF Core?

Tracking (default): EF watches entity changes → SaveChanges() persists. No-Tracking (.AsNoTracking()): read-only, faster, less memory.

Q16. Can we handle transactions in EF Core?

Yes. context.Database.BeginTransactionAsync() or ambient transaction via TransactionScope. SaveChanges() is atomic by default.

Q17. What is explicit transaction?

Manually control begin/commit/rollback for multi-step operations spanning multiple SaveChanges() calls.

Q18. When ADO.NET is available, why do we still need Entity Framework?

EF Core reduces boilerplate (no raw SQL for CRUD), provides migrations, change tracking, LINQ, type safety, faster development. ADO.NET for extreme performance.

Q19. Can we use LINQ with ADO.NET?

LINQ to Objects works on in-memory collections. LINQ to SQL (IQueryable) is EF Core feature — not raw ADO.NET.

Q20. EF Core Fetch by PK — employee with Id = 5

_db.Employees.FindAsync(5) — uses PK, checks cache first. Or _db.Employees.FirstOrDefaultAsync(e => e.Id == 5).

Q21. Select vs Where in LINQ?

Where: filters rows (returns IEnumerable/IQueryable of same type). Select: projects/transforms rows (returns new type). Where = SQL WHERE, Select = SQL SELECT columns.

Q22. Find Method — when use Find vs Where?

Find(key): checks first-level cache, then DB — fastest for PK lookup, only for single entity. Where: flexible, any condition, returns collection.

Q23. EF Core Isolation Levels

Supported via TransactionScope or explicit transaction: ReadUncommitted, ReadCommitted (default), RepeatableRead, Serializable, Snapshot.

Q24. AsNoTracking — what does it mean?

Entities returned are NOT tracked by ChangeTracker. No overhead for change detection. Use for read-only queries for better performance.

Q25. Trade-offs EF Core vs raw ADO.NET?

EF Core: faster dev, migrations, change tracking, LINQ — slower runtime for complex queries. ADO.NET: maximum performance, full control — more code, no migrations.

EF Core Code Examples

QEF1. Complete EF Core Setup Example

Code:

```
// 1. Entity
public class Employee {
    public int    Id       { get; set; }
    public string Name     { get; set; }
    public decimal Salary  { get; set; }
    public int    DeptId   { get; set; }
    public Department Department { get; set; }  // nav property
}

// 2. DbContext
public class AppDbContext : DbContext {
    public AppDbContext(DbContextOptions<AppDbContext> opts) : base(opts) { }
    public DbSet<Employee> Employees { get; set; }
    public DbSet<Department> Departments { get; set; }

    protected override void OnModelCreating(ModelBuilder mb) {
        // Fluent API
        mb.Entity<Employee>()
            .HasOne(e => e.Department)
            .WithMany(d => d.Employees)
            .HasForeignKey(e => e.DeptId);

        mb.Entity<Employee>()
            .Property(e => e.Salary).HasColumnType("decimal(18,2)");
    }
}
```

```
// 3. Register
builder.Services.AddDbContext<AppDbContext>(opt =>
    opt.UseSqlServer(builder.Configuration.GetConnectionString("Default")));

// 4. Commands
// dotnet ef migrations add InitialCreate
// dotnet ef database update

// 5. Usage
// Find by PK (cache-aware)
var emp = await _db.Employees.FindAsync(5);

// Eager loading
var emps = await _db.Employees
    .Include(e => e.Department)
    .Where(e => e.Salary > 50000)
    .AsNoTracking()   // read-only — faster
    .ToListAsync();

// Explicit transaction
using var tx = await _db.Database.BeginTransactionAsync();
try {
    _db.Employees.Add(new Employee { Name = "Vamshi", Salary = 80000 });
    await _db.SaveChangesAsync();
    await tx.CommitAsync();
} catch {
    await tx.RollbackAsync();
}
```

QEF2. Lazy vs Eager Loading Comparison

Code:

```
// Eager Loading — one query with JOIN (preferred)
var orders = await _db.Orders
    .Include(o => o.Customer)
    .Include(o => o.Items).ThenInclude(i => i.Product)
    .ToListAsync();
// SQL: SELECT * FROM Orders JOIN Customers ... JOIN OrderItems JOIN Products ...

// Lazy Loading — N+1 queries (avoid for lists)
// Requires: UseLazyLoadingProxies() + virtual nav props
var orders = await _db.Orders.ToListAsync(); // 1 query
foreach (var o in orders)
    Console.WriteLine(o.Customer.Name); // +1 query per order = N+1 problem!

// Explicit Loading — load on demand manually
var order = await _db.Orders.FindAsync(1);
await _db.Entry(order).Reference(o => o.Customer).LoadAsync();
await _db.Entry(order).Collection(o => o.Items).LoadAsync();
```

SECTION 5: SQL SERVER (61 Questions)

Q1. Explain Clustered vs Non-Clustered Indexes.

Clustered: 1 per table, determines physical row order, data pages at leaf — fast range queries. Non-Clustered: multiple per table, separate B-tree with pointer to data — fast point lookups. Primary key is clustered by default.

Q2. What is an Execution Plan in SQL?

Visual/text representation of how SQL Server executes a query. Shows seeks vs scans, join types, cost estimates. View in SSMS with Ctrl+M or SET STATISTICS IO ON.

Q3. Difference between UNION vs UNION ALL.

UNION: removes duplicates (sorts internally — slower). UNION ALL: keeps all rows (no sort — faster). Use UNION ALL unless duplicates must be removed.

Q4. Explain Normalization in SQL.

Organising tables to reduce redundancy and dependency. 1NF: atomic values. 2NF: no partial dependency. 3NF: no transitive dependency. BCNF: stricter 3NF.

Q5. What is CTE (Common Table Expression)?

Temporary named result set within a query using WITH clause. Improves readability. Supports recursion for hierarchical data.

Q6. What are Stored Procedures vs Functions?

SP: can DML, no return required, call with EXEC, transaction support. Function: must return value, used in SELECT, cannot DML (scalar/table-valued).

Q7. What is SQL Injection and how to prevent it?

Attacker injects malicious SQL via input. Prevent: parameterised queries, stored procedures, ORM, input validation, least privilege.

Q8. Explain Triggers in SQL and their types.

Code that auto-executes on INSERT/UPDATE/DELETE. AFTER trigger: post-operation. INSTEAD OF trigger: replaces operation. Used for auditing, business rules.

Q9. What is Deadlock in SQL? How to prevent it?

Two transactions each waiting for the other's lock — circular wait. Prevent: consistent lock order, short transactions, appropriate isolation level, NOLOCK hint.

Q10. What is Normalization and state its forms?

1NF: atomic values, no repeating groups. 2NF: 1NF + no partial dependency on composite PK. 3NF: 2NF + no transitive dependency. BCNF: 3NF + every determinant is a candidate key.

Q11. What is BOYCE CODD normal form?

Stricter version of 3NF. For every functional dependency X→Y, X must be a superkey. Removes anomalies not covered by 3NF.

Q12. Indexes in SQL

Data structures (B-tree) that speed up SELECT queries at cost of slower writes. Types: Clustered, Non-Clustered, Unique, Filtered, Columnstore, Full-Text.

Q13. Cluster vs Non cluster

See Q1 — same question.

Q14. What is execution plan?

See Q2. Execution plan shows table scans (slow) vs index seeks (fast), hash joins, sort operations, estimated vs actual rows.

Q15. Primary vs Unique

PRIMARY KEY: unique + not null + clustered by default + one per table. UNIQUE: allows one null + non-clustered by default + multiple per table.

Q16. Union vs Union ALL

See Q3.

Q17. ISNULL vs Coalesce

ISNULL(expr, replacement): SQL Server specific, 2 args, returns same type as first arg. COALESCE(a,b,c,...): ANSI standard, multiple args, returns first non-null.

Q18. Truncate vs Delete

TRUNCATE: removes all rows, no WHERE, no triggers, minimal logging, resets identity, faster — cannot rollback in some DBs. DELETE: row-by-row, supports WHERE/triggers, fully logged, can rollback.

Q19. Table vs Views

Table: stores data physically. View: saved SELECT query — virtual table, no storage (except indexed views). Simplifies complex queries, security layer.

Q20. Stored Procedure vs Function

See Q6.

Q21. What is CTE?

See Q5.

Q22. Temporary table vs CTE

Temp table (#temp): physical storage in tempdb, can be reused multiple times in batch, can have indexes. CTE: virtual, exists only for one statement, no storage, cleaner syntax.

Q23. Temporary table vs Table variable

#Temp table: stored in tempdb, visible to sub-procedures, participates in transactions, statistics available. @TableVar: in-memory, local scope, faster for small data.

Q24. Sub Queries vs CTE

Subquery: nested in main query, can be correlated, harder to read. CTE: named, reusable within query, recursive support, more readable.

Q25. SQL Injection and how will you prevent it?

See Q7.

Q26. Triggers in SQL, purpose?

Enforce business rules automatically, audit changes (track before/after), cascade updates/deletes, maintain derived data.

Q27. What is ROW_NUMBER?

Window function assigning sequential integers (1,2,3…) to rows within a partition, ordered by specified column. No gaps, no ties — always unique.

Q28. Rank vs DenseRank

RANK(): ties get same rank, next rank SKIPS (1,1,3). DENSE_RANK(): ties get same rank, NO skip (1,1,2).

Q29. What are Magic tables?

Inserted and Deleted virtual tables inside triggers. Inserted: new values (after INSERT/UPDATE). Deleted: old values (after DELETE/UPDATE).

Q30. What is Deadlock?

See Q9.

Q31. What is WITH (NOLOCK)?

Table hint = READ UNCOMMITTED — reads data without acquiring shared locks. Faster reads but may return dirty/phantom data. Use for reporting on large tables.

Q32. What is isolation level?

Controls visibility of uncommitted data between concurrent transactions. Levels: Read Uncommitted, Read Committed (default), Repeatable Read, Serializable, Snapshot.

Q33. What is transaction and How to implement it?

ACID unit of work. BEGIN TRANSACTION → operations → COMMIT or ROLLBACK. Ensures all or nothing.

Q34. MERGE Statement

Combines INSERT, UPDATE, DELETE in one statement based on a condition. Source vs Target.

Q35. Send unlimited parameters through stored procedure

Use table-valued parameter (TVP) or pass comma-separated string and parse with STRING_SPLIT().

Q36. Difference between SQL vs NoSQL.

SQL: relational, schema, ACID, JOINs — structured data. NoSQL: schema-less, eventual consistency, horizontal scale — unstructured/semi-structured. Types: document (MongoDB), key-value (Redis), column (Cassandra), graph (Neo4j).

Q37. What is partitioning in NoSQL?

Sharding: distributing data across multiple nodes by a partition key. Each node stores a subset. Enables horizontal scaling.

Q38. Explain primary key and secondary index.

Primary key: clustered index, physically orders data, unique identifier. Secondary index: non-clustered, additional lookup structure on non-PK column.

Q39. How does SQL communication happen internally?

Client → TDS protocol → SQL Server → Parser → Query Optimiser → Execution Plan → Storage Engine → Buffer Pool → Data Pages.

Q40. Find the Nth salary

SELECT DISTINCT Salary FROM Employees ORDER BY Salary DESC OFFSET N-1 ROWS FETCH NEXT 1 ROW ONLY; Or DENSE_RANK().

Q41. Find students with same scores

SELECT FirstName, LastName, Score FROM Students WHERE Score IN (SELECT Score FROM Students GROUP BY Score HAVING COUNT(*) > 1);

Q42. Window functions

ROW_NUMBER, RANK, DENSE_RANK, NTILE, LAG, LEAD, SUM OVER, AVG OVER. Operate on a window/partition of rows without collapsing.

Q43. Rank, dense rank

RANK(): gaps after ties. DENSE_RANK(): no gaps. Both need OVER(ORDER BY col).

Q44. Indexes & types

Clustered, Non-Clustered, Unique, Composite, Filtered, Columnstore, Full-Text, Spatial.

Q45. How many cluster indexes per table

Exactly ONE clustered index per table (it IS the table's physical order).

Q46. CTE vs temp table

CTE: one-shot, readable, recursive. Temp table: reusable, can index, survives across multiple statements in same session.

Q47. Scalar Functions — explain the scenario

Returns a single value. Used in SELECT/WHERE. Example: fn_GetFullName(@First,@Last). Can be performance issue if used on large tables (row-by-row).

Q48. Execution Plan or SQL Profiler?

Execution Plan: analyse a specific query's cost and strategy. SQL Profiler / Extended Events: capture all queries over time for performance monitoring.

Q49. What is Common Table Expression — explain the scenario

WITH clause defines named temp result set. Use when same subquery used multiple times, or hierarchical data needed.

Q50. User Defined Function in SQL

Scalar (returns single value), Table-Valued (returns table), Multi-Statement table-valued. Custom reusable logic.

Q51. Segregate array with 0s and 1s without sort

Two-pointer approach: left pointer finds 1, right pointer finds 0, swap. O(n) time, O(1) space.

Q52. Select duplicate records from table

```
SELECT col, COUNT(*) FROM table GROUP BY col HAVING COUNT(*) > 1;
```

Q53. COUNT(*) vs COUNT(name) vs COUNT(address) — which is faster?

COUNT(*): fastest — counts all rows regardless of NULLs. COUNT(col): counts non-NULL values — slightly slower (checks nulls). COUNT(*) is fastest.

Q54. Find max salary from employees

```
SELECT MAX(Salary) FROM Employees;
```

Q55. Find employees having 2nd highest salary

```
SELECT * FROM Employees WHERE Salary = (SELECT MAX(Salary) FROM Employees WHERE Salary < (SELECT MAX(Salary) FROM Employees));
```

Q56. Left join in LINQ

```
// Method: var result = emps.GroupJoin(depts, e => e.DeptId, d => d.Id, (e,ds) => new{e,ds}).SelectMany(x => x.ds.DefaultIfEmpty(), (x,d) => new{x.e.Name, Dept=d?.Name}).ToList();
```

Q57. Query for Nth max salary (80% of interviews)

```
SELECT TOP 1 Salary FROM (SELECT DISTINCT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS R FROM Employees) t WHERE R = @N;
```

Q58. Each Dept with max employees

```
SELECT TOP 1 DeptId, COUNT(*) AS Cnt FROM Employees GROUP BY DeptId ORDER BY Cnt DESC;
```

Q59. Delete duplicate employees

```
DELETE FROM Employees WHERE Id NOT IN (SELECT MIN(Id) FROM Employees GROUP BY Name, Email);
```

Q60. Change true to false false to true using CASE WHEN

```
UPDATE Employees SET IsActive = CASE WHEN IsActive = 1 THEN 0 ELSE 1 END;
```

Q61. Employee with max salary

```
SELECT TOP 1 * FROM Employees ORDER BY Salary DESC;
```

SQL Code Examples

QSQL1. Indexes, CTE, Window Functions, Nth Salary

Code:

```
-- ─── INDEXES ───
CREATE CLUSTERED     INDEX IX_EmpPK   ON Employees(Id);
CREATE NONCLUSTERED  INDEX IX_EmpName ON Employees(LastName, FirstName) INCLUDE (Salary);

-- ─── CTE – simple ───
WITH HighEarners AS (
    SELECT Id, Name, Salary,
           DENSE_RANK() OVER (ORDER BY Salary DESC) AS Rnk
    FROM Employees
)
SELECT * FROM HighEarners WHERE Rnk <= 3;

-- ─── CTE – recursive (org hierarchy) ───
WITH OrgTree AS (
    SELECT Id, Name, ManagerId, 1 AS Level
    FROM Employees WHERE ManagerId IS NULL
    UNION ALL
    SELECT e.Id, e.Name, e.ManagerId, ot.Level + 1
    FROM Employees e JOIN OrgTree ot ON e.ManagerId = ot.Id
)
SELECT * FROM OrgTree ORDER BY Level;

-- ─── Nth highest salary (N=3) ───
SELECT TOP 1 Salary
FROM (SELECT DISTINCT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) R
      FROM Employees) t
WHERE R = 3;

-- ─── ROW_NUMBER / RANK / DENSE_RANK ───
SELECT Name, Salary,
    ROW_NUMBER()  OVER (ORDER BY Salary DESC) AS RowNum,
    RANK()        OVER (ORDER BY Salary DESC) AS Rnk,
    DENSE_RANK()  OVER (ORDER BY Salary DESC) AS DenseRnk
FROM Employees;
```

QSQL2. Stored Procedure, Trigger, Transaction

Code:

```
-- ─── Stored Procedure ───
CREATE PROCEDURE sp_GetEmployeesByDept
    @DeptId INT, @MinSalary DECIMAL = 0
AS BEGIN
    SELECT Id, Name, Salary
    FROM Employees
    WHERE DeptId = @DeptId AND Salary >= @MinSalary
    ORDER BY Salary DESC;
END
EXEC sp_GetEmployeesByDept @DeptId = 5, @MinSalary = 50000;

-- ─── Trigger (audit) ───
CREATE TRIGGER trg_EmpSalaryAudit
ON Employees AFTER UPDATE
AS BEGIN
    INSERT INTO AuditLog (EmployeeId, OldSalary, NewSalary, ChangedAt)
    SELECT d.Id, d.Salary, i.Salary, GETUTCDATE()
    FROM DELETED d JOIN INSERTED i ON d.Id = i.Id
    WHERE d.Salary != i.Salary;
END

-- ─── Transaction ───
```

```
BEGIN TRY
    BEGIN TRANSACTION
        UPDATE Accounts SET Balance -= 500 WHERE Id = 1;  -- debit
        UPDATE Accounts SET Balance += 500 WHERE Id = 2;  -- credit
    COMMIT TRANSACTION
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION
    THROW;
END CATCH

-- ─── MERGE ───
MERGE Employees AS Target
USING NewEmployees AS Source ON Target.Email = Source.Email
WHEN MATCHED     THEN UPDATE SET Target.Name = Source.Name
WHEN NOT MATCHED THEN INSERT (Name, Email) VALUES (Source.Name, Source.Email)
WHEN NOT MATCHED BY SOURCE THEN DELETE;

-- ─── Delete duplicates ───
DELETE FROM Employees
WHERE Id NOT IN (
    SELECT MIN(Id) FROM Employees GROUP BY Email
);
```

QSQL3. SQL Injection Prevention

Code:

```
-- ─── VULNERABLE ───
-- string q = "SELECT * FROM Users WHERE Email = '" + input + "'";
-- Input: ' OR '1'='1 → returns all users!

-- ─── SAFE: parameterised query (ADO.NET) ───
using var cmd = new SqlCommand("SELECT * FROM Users WHERE Email = @email", conn);
cmd.Parameters.AddWithValue("@email", userInput);  // safe

-- ─── SAFE: Stored procedure ───
CREATE PROCEDURE sp_GetUser @Email NVARCHAR(200)
AS BEGIN
    SELECT * FROM Users WHERE Email = @Email; -- parameter, not concatenation
END
EXEC sp_GetUser @Email = 'user@test.com';

-- ─── SAFE: EF Core (always parameterised) ───
var user = await _db.Users.FirstOrDefaultAsync(u => u.Email == email);
// EF generates: WHERE Email = @p0  -- parameterised automatically
```

INTERVIEW ANSWERS — PART 3

Design Patterns · SOLID · Angular · Azure · DevOps · Coding Problems

SECTION 6: DESIGN PATTERNS & ARCHITECTURE (57 Questions)

Q1. Singleton pattern code — write it out

Singleton ensures only ONE instance of a class exists for the entire application.
Common uses: Logger, Configuration, DB connection factory.
Key rule: private constructor so nobody can call new outside the class.

▶ Thread-safe Singleton — two implementations:

```
// ── IMPLEMENTATION 1: Double-Check Locking ──────────────────────
public sealed class AppLogger
{
    // _instance is null at start — created only once
    private static AppLogger _instance;
    // _lock ensures only one thread creates the instance
    private static readonly object _lock = new object();

    // Private ctor — nobody outside can call new AppLogger()
    private AppLogger() { }

    public static AppLogger Instance
    {
        get
        {
            if (_instance == null)           // first check (no lock — fast path)
            {
                lock (_lock)                 // only one thread enters at a time
                {
                    if (_instance == null)   // second check inside lock — safe
                        _instance = new AppLogger();
                }
            }
            return _instance;
        }
    }

    public void Log(string msg) =>
        Console.WriteLine($"[{DateTime.Now:HH:mm:ss}] {msg}");
}

// ── IMPLEMENTATION 2: Lazy<T> (RECOMMENDED — simpler, built-in thread safety) ─
public sealed class Config
{
    // Lazy<T> delays creation until first access AND is thread-safe by default
```

```
    private static readonly Lazy<Config> _lazy =
        new Lazy<Config>(() => new Config());

    public static Config Instance => _lazy.Value;  // first call creates, all others
return same
    private Config() { }

    public string Theme { get; set; } = "Dark";
}

// ── USAGE ──────────────────────────────────────────────────────
AppLogger.Instance.Log("App started");
AppLogger.Instance.Log("Order placed");
// Both calls use the SAME AppLogger instance in memory

Console.WriteLine(Config.Instance.Theme);  // Dark
// Config.Instance always returns the same object
```

💡 Why Lazy<T>? Lazy<T> is thread-safe by default (LazyThreadSafetyMode.ExecutionAndPublication) and reads very cleanly. The lambda runs only once — on the first .Value access.

Q2. What is Design Pattern and why do we need?

A design pattern is a named, reusable solution to a commonly occurring software design problem.
Patterns are NOT copy-paste code — they are templates / blueprints.
They save time because smarter developers already solved the problem.
They give teams a shared vocabulary ("lets use Strategy here").

Q3. Types of Design Pattern — Creational, Structural, Behavioural

CREATIONAL — how objects are created:
Singleton, Factory Method, Abstract Factory, Builder, Prototype

STRUCTURAL — how classes/objects are composed:
Adapter, Decorator, Facade, Proxy, Composite, Bridge, Flyweight

BEHAVIOURAL — how objects communicate:
Strategy, Observer, Command, Iterator, State, Chain of Responsibility, Template Method, Mediator, CQRS

Q4. Which Design patterns have you used in your project?

Repository: data access abstraction — switch DB without changing business logic.
Factory: create the right payment gateway (Stripe/PayPal) based on config.
Strategy: choose discount calculation at runtime without if-else chains.
CQRS + MediatR: separate read and write models in order management.
Decorator: add caching on top of repository without modifying the repository class.
Singleton: one shared AppLogger across the entire application.

Q5. Which Design patterns you know?

Creational: Singleton, Factory, Abstract Factory, Builder.
Structural: Repository (variant of Proxy/Facade), Adapter, Decorator.
Behavioural: Strategy, Observer, Command, Mediator, CQRS.

Q6. What is Singleton DP? Write it down

See Q1 above — complete code provided there.

Q7. Connection pooling in Singleton — why problematic from DB perspective?

Many developers think "put DbContext in Singleton so we reuse one connection."
This is WRONG because DbContext is NOT thread-safe.
Two simultaneous requests will corrupt each other's tracked entity state.
Correct approach: register DbContext as SCOPED (one per HTTP request).
ADO.NET connection pooling happens transparently at the driver level — you do not manage it.

Correct registration:

```
// WRONG — DbContext as Singleton
builder.Services.AddSingleton<AppDbContext>();   // DO NOT DO THIS

// CORRECT — new DbContext per HTTP request
builder.Services.AddDbContext<AppDbContext>(opt =>
    opt.UseSqlServer(connectionString));
// AddDbContext registers as Scoped automatically

// ADO.NET connection pooling is ON by default via the connection string
// "Max Pool Size=100;Min Pool Size=5" — the driver manages this, not your code
```

DbContext holds a ChangeTracker. If shared across threads, Thread A modifying entity A and Thread B modifying entity B will collide — SaveChanges might save wrong data or throw.

Q8. Singleton pattern disadvantage

1. Hard to unit test — global state is shared between tests; must reset between tests.
2. Hidden dependency — class secretly depends on a global, not declared in constructor.
3. Violates Single Responsibility — manages its own lifecycle AND its own logic.
4. Thread safety complexity — need locking code.
5. Cannot easily subclass or swap implementation.
6. Lifetime mismatch — injecting Scoped service into Singleton causes CaptiveDependency bug.

Q9. Breaking the Singleton using direct references, Iterator, IEnumerable

Singleton can be broken (multiple instances created) via:
1. Reflection: call private constructor via ConstructorInfo.Invoke()
2. Serialisation: deserialise creates a new object
3. Clone: if class implements ICloneable improperly
Protect: seal the class, handle serialisation, use Lazy<T>.

Q10. What is Strategy DP?

Strategy defines a FAMILY of algorithms, puts each in its own class, and makes them interchangeable at runtime.
Problem it solves: avoid big if-else / switch blocks when behaviour needs to vary.
Real world: payment methods, sorting algorithms, discount rules, export formats.

Strategy Pattern — Discount Calculation:

```
// Step 1: Define the strategy interface (the CONTRACT)
public interface IDiscountStrategy
{
    decimal Apply(decimal originalPrice);
}

// Step 2: Concrete strategies — each algorithm in its own class
public class NoDiscount : IDiscountStrategy
{
    public decimal Apply(decimal price) => price;   // nothing removed
}

public class FestivalDiscount : IDiscountStrategy
{
```

```
    public decimal Apply(decimal price) => price * 0.80m;  // 20% off
}

public class ClearanceDiscount : IDiscountStrategy
{
    public decimal Apply(decimal price) => price * 0.50m;  // 50% off
}

// Step 3: Context — holds strategy, delegates to it
public class ShoppingCart
{
    private IDiscountStrategy _discount = new NoDiscount(); // default

    public void SetDiscount(IDiscountStrategy strategy) => _discount = strategy;

    public decimal Checkout(decimal total)
    {
        decimal finalPrice = _discount.Apply(total);
        Console.WriteLine($"Original: {total}, Final: {finalPrice}");
        return finalPrice;
    }
}

// Step 4: Usage — swap strategy without changing Cart
var cart = new ShoppingCart();
cart.Checkout(1000);                          // 1000 (NoDiscount)

cart.SetDiscount(new FestivalDiscount());
cart.Checkout(1000);                          // 800  (20% off)

cart.SetDiscount(new ClearanceDiscount());
cart.Checkout(1000);                          // 500  (50% off)
```

Key benefit: adding "VIPDiscount : IDiscountStrategy" adds ZERO changes to ShoppingCart — Open/Closed principle. Without Strategy, ShoppingCart would have if/switch that grows every time a new discount type is added.

Q11. What is Adapter DP?

Adapter makes an incompatible interface work with client code.
Analogy: a power plug adapter lets a 3-pin plug work in a 2-pin socket.
Real world: wrapping a legacy class, third-party library, or external API so it matches your internal interface.

Adapter Pattern — Legacy SMS integration:

```
// Our application expects this interface (the TARGET)
public interface IMessageSender
{
    void Send(string to, string message);
}

// Third-party legacy SMS library we cannot modify (the ADAPTEE)
public class LegacySmsLib
{
    // Different method signature — incompatible!
    public void SendSms(string phoneNumber, string text, bool isUrgent, string senderId)
    {
        Console.WriteLine($"SMS to {phoneNumber}: {text}");
    }
}

// ADAPTER — wraps the incompatible class, exposes our interface
public class SmsAdapter : IMessageSender
{
    private readonly LegacySmsLib _legacy;

    public SmsAdapter()
```

```
    {
        _legacy = new LegacySmsLib();  // owns the adaptee
    }

    // Translate our simple call into the legacy library's complex signature
    public void Send(string to, string message)
    {
        _legacy.SendSms(to, message, isUrgent: false, senderId: "MYAPP");
    }
}

// Client code — only knows about IMessageSender, unaware of legacy lib
public class NotificationService
{
    private readonly IMessageSender _sender;

    public NotificationService(IMessageSender sender) => _sender = sender;

    public void NotifyUser(string phone, string msg) => _sender.Send(phone, msg);
}

// Wiring
IMessageSender sender = new SmsAdapter();
var service = new NotificationService(sender);
service.NotifyUser("+1234567890", "Your order has shipped!");
```

If the SMS provider changes, you write a new adapter — NotificationService does not change. This is Dependency Inversion in action.

Q12. What is Abstract Factory DP?

Abstract Factory creates FAMILIES of related objects without specifying their concrete classes.

Difference from Factory Method: Factory creates ONE product; Abstract Factory creates a SUITE of related products.

Real world: UI toolkit (Windows vs Mac buttons+checkboxes), cloud provider (Azure vs AWS storage+messaging).

Abstract Factory — Cross-platform UI:

```
// Product interfaces
public interface IButton   { void Render(); }
public interface ICheckbox { void Render(); }

// Abstract Factory interface — creates a FAMILY of related products
public interface IUIFactory
{
    IButton   CreateButton();
    ICheckbox CreateCheckbox();
}

// Concrete products for Windows
public class WinButton   : IButton   { public void Render() => Console.WriteLine("Windows
Button"); }
public class WinCheckbox : ICheckbox { public void Render() => Console.WriteLine("Windows
Checkbox"); }

// Concrete products for Mac
public class MacButton   : IButton   { public void Render() => Console.WriteLine("Mac
Button"); }
public class MacCheckbox : ICheckbox { public void Render() => Console.WriteLine("Mac
Checkbox"); }

// Concrete factories — each creates a MATCHING family
public class WindowsFactory : IUIFactory
{
    public IButton   CreateButton()   => new WinButton();    // always Windows style
```

```
    public ICheckbox CreateCheckbox() => new WinCheckbox();
}

public class MacFactory : IUIFactory
{
    public IButton   CreateButton()   => new MacButton();    // always Mac style
    public ICheckbox CreateCheckbox() => new MacCheckbox();
}

// Application — only uses factory interface, unaware of concrete types
public class App
{
    private readonly IButton   _button;
    private readonly ICheckbox _checkbox;

    public App(IUIFactory factory)          // inject the factory
    {
        _button   = factory.CreateButton();
        _checkbox = factory.CreateCheckbox();
    }

    public void RenderUI() { _button.Render(); _checkbox.Render(); }
}

// Choose factory at startup based on OS
IUIFactory factory = OperatingSystem.IsWindows() ? new WindowsFactory() : new
MacFactory();
var app = new App(factory);
app.RenderUI();   // all components are from the SAME family — consistent look
```

Without Abstract Factory you might mix Windows button with Mac checkbox. The factory guarantees consistency of the product family.

Q13. What is Repository DP? List its advantages

Repository hides all data-access logic behind an interface.
Business logic talks to the interface — it does not know (or care) if data comes from SQL, Cosmos DB, or a file.
Advantages:

1. Testable — mock IRepository in unit tests without a real database.
2. Swappable — change from SQL to Mongo by swapping implementation, zero business logic change.
3. Single place — query logic lives in one class, not scattered.
4. Consistent API — Add, GetById, GetAll, Update, Delete everywhere.

Repository Pattern — full example:

```
// Step 1: Define interface (the CONTRACT)
public interface IProductRepository
{
    Task<Product?>            GetByIdAsync(int id);
    Task<IEnumerable<Product>> GetAllAsync();
    Task                       AddAsync(Product product);
    Task                       UpdateAsync(Product product);
    Task                       DeleteAsync(int id);
}

// Step 2: EF Core implementation
public class EfProductRepository : IProductRepository
{
    private readonly AppDbContext _db;
    public EfProductRepository(AppDbContext db) => _db = db;

    // FindAsync checks first-level cache, then queries DB
    public async Task<Product?> GetByIdAsync(int id) => await _db.Products.FindAsync(id);

    // AsNoTracking = read-only, no change-tracking overhead
```

```
    public async Task<IEnumerable<Product>> GetAllAsync() =>
        await _db.Products.AsNoTracking().ToListAsync();

    public async Task AddAsync(Product p)    { _db.Products.Add(p);    await
_db.SaveChangesAsync(); }
    public async Task UpdateAsync(Product p) { _db.Products.Update(p); await
_db.SaveChangesAsync(); }
    public async Task DeleteAsync(int id)
    {
        var p = await GetByIdAsync(id);
        if (p != null) { _db.Products.Remove(p); await _db.SaveChangesAsync(); }
    }
}

// Step 3: Register (Scoped so each request gets its own instance)
builder.Services.AddScoped<IProductRepository, EfProductRepository>();

// Step 4: Use in service — no EF Core knowledge here
public class ProductService
{
    private readonly IProductRepository _repo;
    public ProductService(IProductRepository repo) => _repo = repo;

    public async Task<Product?> GetProduct(int id) => await _repo.GetByIdAsync(id);
}

// Step 5: Unit test — no real DB needed
var mockRepo = new Mock<IProductRepository>();
mockRepo.Setup(r => r.GetByIdAsync(1)).ReturnsAsync(new Product { Id = 1, Name = "Laptop"
});
var service  = new ProductService(mockRepo.Object);
var result   = await service.GetProduct(1);   // fast, no DB
```

The service never imports Entity Framework — it only knows about IProductRepository. If tomorrow you move to MongoDB, you write MongoProductRepository and change one line of DI registration.

Q14. What is CQRS DP?

CQRS = Command Query Responsibility Segregation.

Separate reads (Queries) from writes (Commands) into completely different models.

Query: returns data, never changes state.

Command: changes state, returns minimal data (e.g. new ID).

Why: read and write loads are different — reads can be 100x more frequent; each side can be optimised independently.

CQRS with MediatR:

```
// ── COMMAND: changes state ──────────────────────────────────
// Record = immutable DTO; ideal for commands
public record CreateOrderCommand(int CustomerId, List<int> ProductIds)
    : IRequest<int>;   // returns new Order ID

// Handler: does the actual write
public class CreateOrderHandler : IRequestHandler<CreateOrderCommand, int>
{
    private readonly IOrderRepository _repo;
    public CreateOrderHandler(IOrderRepository repo) => _repo = repo;

    public async Task<int> Handle(CreateOrderCommand cmd, CancellationToken ct)
    {
        var order = new Order
        {
            CustomerId = cmd.CustomerId,
            Items      = cmd.ProductIds.Select(id => new OrderItem { ProductId = id
}).ToList(),
            CreatedAt  = DateTime.UtcNow
```

```
        };
        await _repo.AddAsync(order);
        return order.Id;   // only the new ID goes back — not the full object
    }
}

// ── QUERY: returns data, zero side effects ──────────────────────
public record GetOrderQuery(int OrderId) : IRequest<OrderDto?>;

public class GetOrderHandler : IRequestHandler<GetOrderQuery, OrderDto?>
{
    private readonly IOrderReadDb _readDb;   // could be a read replica or Redis
    public GetOrderHandler(IOrderReadDb readDb) => _readDb = readDb;

    public async Task<OrderDto?> Handle(GetOrderQuery q, CancellationToken ct) =>
        await _readDb.GetOrderDtoAsync(q.OrderId);
}

// ── Controller: thin, dispatches via MediatR ────────────────────
[ApiController, Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly IMediator _mediator;
    public OrdersController(IMediator mediator) => _mediator = mediator;

    [HttpPost]
    public async Task<IActionResult> Create([FromBody] CreateOrderCommand cmd)
    {
        int newId = await _mediator.Send(cmd);
        return CreatedAtAction(nameof(Get), new { id = newId }, new { id = newId });
    }

    [HttpGet("{id}")]
    public async Task<IActionResult> Get(int id)
    {
        var dto = await _mediator.Send(new GetOrderQuery(id));
        return dto is null ? NotFound() : Ok(dto);
    }
}
```

MediatR is the "postman" — it receives a request object and finds the registered handler. The controller has zero business logic, making it very easy to test each handler in isolation.

Q15. What are 3 important elements of CQRS?

1. COMMAND — intent to mutate state (CreateOrder, CancelFlight). Carries the input data.
2. QUERY — request for data (GetOrder, ListFlights). Must not change anything.
3. HANDLER — processes the command or query. One handler per command/query. Contains the actual logic.

Q16. Why is MediatR in CQRS?

MediatR is the MEDIATOR / message bus within the process.
Without it: Controller → imports OrderService → imports PaymentService → tight coupling.
With MediatR: Controller → sends Command → MediatR finds handler → handler runs.
The controller never imports any service directly — it only knows IMediator.
Also enables: pipeline behaviours (logging, validation, caching) that wrap every request transparently.

Q17. Why CQRS is important in Microservices

In microservices each service has its own DB. Reads and writes have very different scaling needs.
Write side: strong consistency, relational DB, lower throughput.
Read side: high throughput, can serve from a denormalised read model / Redis / Elasticsearch.

CQRS lets each side scale independently. Write DB handles 100 req/s; read replica handles 10,000 req/s.

Q18. What is event sourcing in CQRS?

Instead of storing the CURRENT state of a record, you store every EVENT that led to that state.
State is rebuilt by replaying events from beginning.
OrderCreated → ItemAdded → DiscountApplied → OrderShipped → OrderDelivered.
Benefits: full audit trail, time travel (state at any point in time), easier CQRS read-model rebuilding.

Q19. Is SQL Server an Event store?

Not by default, but it CAN be used as one by creating an Events table.
Purpose-built event stores: EventStoreDB (free, open-source), Azure Event Hub + Cosmos DB.
SQL Server as event store: store EventType, Payload (JSON), AggregateId, Timestamp, Version.
Trade-off: SQL lacks stream-native queries; EventStoreDB has built-in projections.

Q20. Have you worked on Clean Architecture?

Clean Architecture = 4 concentric circles:
1. Domain (innermost): Entities, Value Objects, Domain Events, interfaces — ZERO external dependencies.
2. Application: Use Cases, CQRS Handlers, DTOs — depends only on Domain.
3. Infrastructure: EF Core, SQL, Azure, Email — depends on Application interfaces.
4. Presentation: API Controllers, minimal API — depends on Application.
Dependency rule: arrows point INWARD only. Domain knows nothing about EF Core or HTTP.

Q21. What are Microservices and their advantages and disadvantages?

Microservices: independently deployable services each owning one business capability and its own DB.

Advantages:
- Each service scales independently (booking scales without scaling user service)
- Independent deployment (deploy one service without touching others)
- Technology diversity (Node.js for real-time, .NET for booking, Python for ML)
- Fault isolation (payment outage does not bring down browsing)

Disadvantages:
- Distributed system complexity (network calls, partial failures)
- Eventual consistency instead of ACID transactions
- Harder debugging (traces span multiple services)
- Operational overhead (many deployments, many logs, many health checks)

Q22. Monolithic vs Microservices

Monolith: all code in one codebase, one deployment, one DB. Simple ops. Scale EVERYTHING together. Fast initially.
Microservices: many codebases, many deployments, each service owns its DB. Complex ops. Scale what needs scaling. Better for large teams.
Rule of thumb: start monolith, extract to microservices when team/load demands it.

Q23. Why do we use an API Gateway?

Without gateway: clients talk to 10 services at 10 different URLs → auth in every service.
With gateway: one URL, one auth check, one SSL cert, one rate limiter.
Provides: routing, auth, rate limiting, SSL, request aggregation, caching, logging.
Examples: Azure API Management (APIM), Kong, Ocelot (.NET library), AWS API Gateway.

Q24. What is an API Gateway and how does it work?

Client sends request → Gateway authenticates JWT → Gateway routes to correct microservice → aggregates if needed → returns response.

In Azure: APIM sits in front of all APIs. Policies apply: validate-jwt, rate-limit, ip-filter, set-header, transform-body.

Q25. Difference between rate limiting and load balancing

Rate Limiting: control how many requests a SINGLE CLIENT can send per time window. Prevents abuse.

Load Balancing: distribute requests across MULTIPLE SERVER INSTANCES for availability and performance.

They solve different problems — both are usually present together.

Q26. Where should rate limiting be applied?

API Gateway level (first defence — before traffic reaches services).

Or ASP.NET Core 7+ built-in middleware: services.AddRateLimiter() with policies.

For API key-based: per-key limits. For public endpoints: per-IP limits.

Rate Limiting in ASP.NET Core 7+:

```
builder.Services.AddRateLimiter(opt =>
{
    // Fixed window: max 100 requests per 1 minute per client
    opt.AddFixedWindowLimiter("fixed", cfg =>
    {
        cfg.PermitLimit         = 100;
        cfg.Window              = TimeSpan.FromMinutes(1);
        cfg.QueueProcessingOrder = QueueProcessingOrder.OldestFirst;
        cfg.QueueLimit          = 10;   // queue 10 extra, reject rest with 429
    });

    // Return 429 Too Many Requests with helpful headers
    opt.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
});

app.UseRateLimiter();

// Apply to endpoint
app.MapGet("/api/search", SearchHandler).RequireRateLimiting("fixed");
```

Q27. How do you prevent abuse using rate limiting?

Return 429 Too Many Requests with Retry-After header.

Per-IP limits for anonymous users.

Per-API-key limits for authenticated users.

Sliding window policy smooths out burst traffic.

Exponential backoff: clients should wait progressively longer after each 429.

Q28. How do microservices communicate?

SYNCHRONOUS (request-response, tight coupling):

HTTP/REST: simple, widely supported, latency adds up with chain calls.

gRPC: binary protocol, typed contracts (Protobuf), faster than JSON, ideal for service-to-service.

ASYNCHRONOUS (event-driven, loose coupling):

Azure Service Bus: guaranteed delivery, dead-letter, retry, ordering.

RabbitMQ: open-source, flexible routing, pub-sub.

Kafka: high-throughput event streaming, consumer groups, replay.

Q29. How does RabbitMQ communication work?

Producer publishes message to an EXCHANGE.
Exchange routes to one or more QUEUES based on binding rules and routing key.
Consumer reads from queue and ACKs (acknowledges) successful processing.
If consumer crashes without ACK, message goes back to queue for retry.
Dead-Letter Exchange (DLX): failed messages after max retries are routed there for investigation.

RabbitMQ .NET example:

```
// Producer
var factory    = new ConnectionFactory { HostName = "localhost" };
using var conn = factory.CreateConnection();
using var ch2  = conn.CreateModel();

// Declare queue so it exists before publishing
ch2.QueueDeclare(queue: "orders", durable: true, exclusive: false, autoDelete: false);

var body = Encoding.UTF8.GetBytes(JsonSerializer.Serialize(new { OrderId = 42 }));

// Persistent delivery — survives RabbitMQ restart
ch2.BasicPublish(exchange: "", routingKey: "orders",
    basicProperties: ch2.CreateBasicProperties() with { Persistent = true },
    body: body);

// Consumer
var consumer = new EventingBasicConsumer(ch2);
consumer.Received += (_, ea) =>
{
    var msg = Encoding.UTF8.GetString(ea.Body.ToArray());
    Console.WriteLine($"Received: {msg}");
    ch2.BasicAck(ea.DeliveryTag, false);   // ACK = I processed this, remove from queue
    // If processing fails: ch2.BasicNack(ea.DeliveryTag, false, requeue: true);
};
ch2.BasicConsume(queue: "orders", autoAck: false, consumer: consumer);
```

autoAck:false means the message stays in queue until you call BasicAck. This guarantees at-least-once delivery even if the consumer crashes mid-processing.

Q30. Layer vs Tiered

LAYER: logical separation of code within one deployment.
Presentation layer → Business layer → Data layer (all in one exe/dll).

TIER: physical separation — runs on different machines.
Web server (Presentation tier) → App server (Business tier) → Database server (Data tier).

A 3-layer monolith has 3 layers but only 1 tier. A cloud app can have 10 tiers.

Q31. Tier and N tier difference

N-Tier adds more physical tiers as needed:
CDN tier (static assets) → Load Balancer tier → API Gateway tier → App Server tier → Cache tier (Redis) → DB tier → Message Queue tier.
Each tier is independently scalable and maintainable.

Q32. How does one microservice call another internally?

HTTP (sync) and Service Bus (async):

```
// ── SYNC: Named HttpClient with Polly retry ──────────────────
builder.Services.AddHttpClient("InventoryService",
    c => c.BaseAddress = new Uri("http://inventory-svc/"))  // K8s DNS name
    .AddTransientHttpErrorPolicy(p =>
```

```
        p.WaitAndRetryAsync(3, retryAttempt =>
            TimeSpan.FromSeconds(Math.Pow(2, retryAttempt))));  // 2s, 4s, 8s

// Usage in service
public class OrderService
{
    private readonly IHttpClientFactory _factory;
    public OrderService(IHttpClientFactory factory) => _factory = factory;

    public async Task<bool> ReserveStock(int productId, int qty)
    {
        var client = _factory.CreateClient("InventoryService");
        // If this call fails, Polly retries automatically
        var resp = await client.PostAsJsonAsync("api/stock/reserve",
                    new { productId, qty });
        return resp.IsSuccessStatusCode;
    }
}

// ── ASYNC: Azure Service Bus (fire and forget) ──────────────────
public class PaymentService
{
    private readonly ServiceBusSender _sender;

    public async Task PublishPaymentConfirmed(int orderId)
    {
        var payload = new ServiceBusMessage(
            BinaryData.FromObjectAsJson(new { orderId, status = "Paid" }));

        // Producer does not wait for OrderService to process it
        await _sender.SendMessageAsync(payload);
    }
}
```

Use HTTP for: queries needing immediate response (check stock before booking). Use Service Bus for: notifications, side effects (send email after payment) — decouples services so a slow consumer does not block the producer.

Q33. Strategic DDD vs Tactical DDD.

Strategic DDD: high-level — WHERE to draw boundaries.
Bounded Contexts, Context Maps, Subdomains, Ubiquitous Language.
Done with domain experts BEFORE writing code.

Tactical DDD: implementation-level — HOW to write code inside a bounded context.
Entities, Value Objects, Aggregates, Repositories, Domain Services, Domain Events.

Q34. What's a domain and sub domain?

Domain: the entire problem space (e.g., airline ticketing).
Subdomain: smaller problem areas within (Booking, Pricing, Loyalty, Ground Operations).
Software maps one service/bounded context per subdomain ideally.

Q35. Types of sub domain

CORE subdomain: unique competitive advantage — invest most here (e.g., dynamic pricing algorithm).
SUPPORTING subdomain: custom but not differentiating — build lean (e.g., employee scheduling).
GENERIC subdomain: commodity — buy off the shelf (e.g., email sending, PDF generation).

Q36. Bounded context, context map and Ubiquitous Language (UL)

BOUNDED CONTEXT: explicit boundary where a model and UL are consistent.
"Order" means different things in Sales context vs Fulfilment context — separate models.
CONTEXT MAP: diagram showing how bounded contexts relate.
Patterns: Shared Kernel, Customer-Supplier, Anti-Corruption Layer, Conformist.
UBIQUITOUS LANGUAGE: vocabulary shared by devs AND domain experts, used in code AND conversation.

Q37. What is a SAGA pattern?

Manages distributed transactions without 2-phase commit.
Each step has a FORWARD action and a COMPENSATING action (undo).
CHOREOGRAPHY: services react to events from each other — decentralised.
ORCHESTRATION: one saga orchestrator class calls each service in sequence — centralised, easier to reason about.

Q38. What is DDD?

Domain-Driven Design: design software by deeply modelling the business domain.
Key ideas: Ubiquitous Language, Bounded Contexts, Rich domain model (business logic in entities, not anemic).
Avoid: putting all logic in services/controllers with entities as dumb data bags.

Q39. Entity vs Value Object vs Service

ENTITY: has unique identity (OrderId=42), mutable, tracked over time.
VALUE OBJECT: defined by attributes, no identity, immutable — Money(100, USD), Address.
DOMAIN SERVICE: logic that does not naturally belong to an entity — TaxCalculator, CurrencyConverter.

Q40. Aggregate root

An aggregate is a cluster of entities/VOs treated as one unit.
The ROOT is the only entry point — external code references the root only, never inner entities.
Enforces invariants: Order.AddItem() validates max items, not caller.

Q41. CQRS Pattern

See Q14 above — full code example with MediatR provided.

Q42. Repository

See Q13 above — full code example provided.

Q43. Queues

Async buffer between producer and consumer services.
Producer puts message in — continues immediately.
Consumer reads at its own pace — independently scalable.
Azure Service Bus (enterprise), RabbitMQ (open-source), Kafka (high-throughput streaming).

Q44. Draw infrastructure diagram

Client → CDN (static) → Azure Front Door (WAF/SSL) → APIM (auth/rate-limit) → App Service / AKS (microservices) → Redis (cache) → Service Bus (async) → SQL DB / Cosmos DB → Key Vault → App Insights.

Q45. Project & Architecture (Authenticity Check)

Be honest about your actual project. Prepare: 2-min pitch (problem solved, tech stack, your role, challenges). Expect follow-up on specifics — do not exaggerate.

Q46. Explain your current project end-to-end architecture.

Structure your answer: Business problem → Architecture style → Services involved → Data flow → Security → Deployment → Monitoring. Be specific about YOUR contribution.

Q47. Walk through projects in CV with tech justification.

For each project: What was built → Key technical decisions → Why that tech was chosen → Challenges you solved → What you would do differently.

Q48. What technologies were used and why?

Be specific. "Azure Service Bus because we needed guaranteed delivery and dead-letter queue for failed payment notifications." Not just "we used Service Bus."

Q49. System design of MakeMyTrip / Booking system.

Search (Elasticsearch, cached) → Inventory (flight/hotel availability) → Pricing (dynamic, real-time) → Booking (ACID transaction) → Payment (idempotent, idempotency key) → Notification (async via queue) → Loyalty.

Q50. All major microservices in online booking system.

User, Auth, Search, Inventory, Pricing, Booking, Payment, Notification, Recommendation, Reviews, Loyalty/Rewards, Admin/Reporting.

Q51. Scenario-based questions on online booking system failures.

Booking created but payment fails → Saga compensation: cancel booking. Payment succeeds but notification fails → Retry queue. Inventory shows available but actually sold out → Optimistic locking + Saga compensation.

Q52. How to avoid duplicate payments in a booking system?

IDEMPOTENCY KEY: client generates UUID per attempt. API stores key in DB. Second call with same key returns first result without re-charging. Redis with TTL for quick lookup.

Q53. How to handle fault occurrence in an online booking system?

Circuit breaker (Polly) for downstream service outages. Dead-letter queue for unprocessable messages. Saga compensating transactions. Health check endpoints. Fallback responses (show "temporarily unavailable" not crash).

Q54. How can API return fastest search results in MakeMyTrip-like system?

Redis cache for popular routes (city-pair, date). Elasticsearch for full-text/faceted search. Pre-compute prices for common searches. CDN for static. Async prefetch on hover. Connection pooling. Pagination. Compressed responses.

Q55. List project layers in a large-scale application.

UI (Angular/React) → BFF / API Gateway → Application Microservices (Controllers/Handlers) → Domain Layer → Infrastructure (EF Core, Repos, External APIs) → Cross-cutting (Logging, Auth, Cache).

Q56. Singleton and pattern used in your project.

Singleton: AppLogger, Configuration manager. Repository: all data access. Factory: payment gateway selection. Strategy: discount/pricing algorithms. CQRS with MediatR: order and booking modules.

Q57. Any idea regarding NGRX / Redux store in Angular?

NgRx: Redux pattern for Angular. STORE holds state. ACTIONS describe events. REDUCERS are pure functions (state, action) → newState. SELECTORS read slices. EFFECTS handle async (HTTP calls). Use for complex shared state — cart, auth, notifications.

SECTION 7: SOLID PRINCIPLES (18 Questions)

Q1. What is SOLID principle and why implement?

Five design principles that make object-oriented code maintainable, extensible, and testable.
S — Single Responsibility Principle
O — Open/Closed Principle
L — Liskov Substitution Principle
I — Interface Segregation Principle
D — Dependency Inversion Principle
Why: code that violates SOLID becomes hard to change (change one thing, break five others), hard to test (everything is coupled), and hard to extend (must modify existing code).

Q2. State all the principles with example

▶ All 5 SOLID Principles with Real Code:

```
// ── S: Single Responsibility — one reason to change ──────────────
// WRONG: one class doing everything
public class UserManager
{
    public void CreateUser(string name) { /* DB insert */ }
    public void SendWelcomeEmail(string email) { /* SMTP */ }
    public void LogActivity(string msg) { /* File IO */ }
    // If email provider changes → must touch UserManager. Bad.
}

// CORRECT: each class has one job
public class UserService   { public void CreateUser(string name) { } }
public class EmailService  { public void SendWelcomeEmail(string email) { } }
public class ActivityLog   { public void Log(string msg) { } }

// ── O: Open/Closed — extend without modifying ────────────────────
// WRONG: must edit this class for every new shape
public class AreaCalcOld
{
    public double Calc(object shape)
    {
        if (shape is Circle c) return Math.PI * c.R * c.R;  // edit here for new shapes
        throw new NotImplementedException();
    }
}

// CORRECT: each shape knows its own area; AreaCalc never changes
public interface IShape { double Area(); }
public class Circle    : IShape { public double R; public double Area() => Math.PI * R *
R; }
public class Rectangle : IShape { public double W, H; public double Area() => W * H; }
public class Triangle  : IShape { public double B, H; public double Area() => 0.5 * B *
H; }
public class AreaCalc  { public double Calc(IShape s) => s.Area(); }  // never changes

// ── L: Liskov Substitution — child must honour parent contract ────
// WRONG: Penguin IS-A Bird but cannot Fly — breaks caller assumptions
```

```
public class Bird    { public virtual void Fly() { Console.WriteLine("Flying"); } }
public class Penguin : Bird
{
    public override void Fly() => throw new NotSupportedException("I can't fly!");
    // Code that calls bird.Fly() BREAKS when given a Penguin
}

// CORRECT: restructure with capability interfaces
public interface IFlyable  { void Fly(); }
public interface ISwimmable{ void Swim(); }
public class Eagle   : IFlyable  { public void Fly()  => Console.WriteLine("Soaring"); }
public class Penguin2: ISwimmable{ public void Swim() => Console.WriteLine("Swimming"); }

// ── I: Interface Segregation — small focused interfaces ──────────
// WRONG: classes forced to implement methods they don't need
public interface IWorker { void Work(); void Eat(); void Sleep(); }
public class RobotWorker : IWorker
{
    public void Work()  { /* actual work */ }
    public void Eat()   => throw new NotImplementedException(); // robots don't eat!
    public void Sleep() => throw new NotImplementedException(); // or sleep!
}

// CORRECT: split into focused interfaces
public interface IWorkable { void Work(); }
public interface IFeedable { void Eat(); }
public interface ISleepable{ void Sleep(); }
public class HumanWorker : IWorkable, IFeedable, ISleepable
{
    public void Work()  { }
    public void Eat()   { }
    public void Sleep() { }
}
public class RobotWorker2 : IWorkable { public void Work() { } }  // only what it needs

// ── D: Dependency Inversion — depend on abstractions ─────────────
// WRONG: high-level module directly imports low-level concrete class
public class OrderService_Bad
{
    private SqlOrderRepository _repo = new SqlOrderRepository(); // direct, hard-wired
    public void Place(Order o) => _repo.Save(o);
}

// CORRECT: depend on interface, inject concrete via DI
public interface IOrderRepository { Task SaveAsync(Order o); }
public class OrderService_Good
{
    private readonly IOrderRepository _repo;
    public OrderService_Good(IOrderRepository repo) => _repo = repo; // injected
    public async Task Place(Order o) => await _repo.SaveAsync(o);
}
// Swap SQL to Mongo: just change DI registration — OrderService unchanged
```

Memorise the acronym: SRP = "one job", OCP = "extend not edit", LSP = "child behaves as parent promises", ISP = "small contracts", DIP = "depend on abstractions".

Q3. What is Liskov Substitution Principle — explain with example

If S is a subtype of T, objects of type T can be replaced with S without breaking the program.
Simply: code that works with the BASE class must also work correctly with any CHILD class.
Violation symptom: child class throws NotImplementedException or does nothing for an inherited method.

LSP violation vs correct design:

```
// VIOLATION
class Rectangle2
{
```

```
    public virtual int Width  { get; set; }
    public virtual int Height { get; set; }
    public int Area() => Width * Height;
}

class Square : Rectangle2
{
    // Forces both sides equal — violates caller expectations
    public override int Width  { set { base.Width = base.Height = value; } get =>
base.Width; }
    public override int Height { set { base.Width = base.Height = value; } get =>
base.Height; }
}

void PrintArea(Rectangle2 r)
{
    r.Width = 5; r.Height = 10;
    Console.WriteLine(r.Area());  // Expected 50
}

PrintArea(new Rectangle2()); // 50  ✓
PrintArea(new Square());     // 25  ✗  — Square silently changed Width when we set
Height!

// CORRECT: model what things ARE, not inheritance hierarchy
public interface IShape2 { int Area(); }
public class Rect   : IShape2 { public int W, H; public int Area() => W * H; }
public class Square2: IShape2 { public int S;    public int Area() => S * S; }
// Both implement IShape2 — no surprising behaviour
```

The key question: "Can I replace a base class object with a child class object without the caller noticing?" If yes — LSP holds.

Q4. Difference between Open/Closed and Liskov principle

OCP: about EXTENSIBILITY — add new behaviour without modifying existing code. Done by using interfaces/abstract classes so new types can be plugged in.

LSP: about CORRECTNESS of substitution — when you DO subclass, the child must honour the parent's promises. No surprise exceptions, no weaker behaviour.

OCP says "how to add new things safely". LSP says "make sure the new thing actually works the same way".

Q5. Difference between Liskov vs ISP?

LSP: about CLASS inheritance correctness — child must be substitutable for parent.

ISP: about INTERFACE design — clients should not be forced to implement methods they do not use.

LSP is violated when: child throws NotImplemented or weakens a method.

ISP is violated when: one fat interface forces classes to implement irrelevant methods.

Q6. How do you design API for different payment gateways?

Strategy + Factory + DI for payment gateways:

```
// Step 1: Define the payment contract (ISP — focused interface)
public interface IPaymentGateway
{
    Task<PaymentResult> ChargeAsync(PaymentRequest request);
    Task<RefundResult>  RefundAsync(string transactionId, decimal amount);
}

// Step 2: Implement per gateway (OCP — add new gateway without changing existing)
public class StripeGateway : IPaymentGateway
{
```

```
    private readonly IConfiguration _config;
    public StripeGateway(IConfiguration config) => _config = config;

    public async Task<PaymentResult> ChargeAsync(PaymentRequest req)
    {
        // Stripe SDK call — hidden from caller
        Console.WriteLine($"Stripe: charging {req.Amount} {req.Currency}");
        return new PaymentResult { Success = true, TransactionId =
Guid.NewGuid().ToString() };
    }
    public async Task<RefundResult> RefundAsync(string txId, decimal amount)
    {
        Console.WriteLine($"Stripe: refunding {amount} for tx {txId}");
        return new RefundResult { Success = true };
    }
}

public class RazorpayGateway : IPaymentGateway
{
    public async Task<PaymentResult> ChargeAsync(PaymentRequest req) { /* Razorpay SDK */
return new(); }
    public async Task<RefundResult>  RefundAsync(string txId, decimal amount) { return
new(); }
}

// Step 3: Factory creates the right gateway (Strategy pattern)
public class PaymentGatewayFactory
{
    private readonly IServiceProvider _sp;
    public PaymentGatewayFactory(IServiceProvider sp) => _sp = sp;

    public IPaymentGateway Create(string provider) => provider.ToLower() switch
    {
        "stripe"   => _sp.GetRequiredService<StripeGateway>(),
        "razorpay" => _sp.GetRequiredService<RazorpayGateway>(),
        _          => throw new NotSupportedException($"Unknown provider: {provider}")
    };
}

// Step 4: Service uses interface — unaware of concrete gateway (DIP)
public class CheckoutService
{
    private readonly PaymentGatewayFactory _factory;
    public CheckoutService(PaymentGatewayFactory factory) => _factory = factory;

    public async Task<PaymentResult> ProcessPayment(string provider, PaymentRequest req)
    {
        IPaymentGateway gateway = _factory.Create(provider);
        return await gateway.ChargeAsync(req);   // works for any gateway
    }
}

// Step 5: Register gateways
builder.Services.AddScoped<StripeGateway>();
builder.Services.AddScoped<RazorpayGateway>();
builder.Services.AddScoped<PaymentGatewayFactory>();
```

Adding PayPal gateway = add PayPalGateway class + one line in factory switch. CheckoutService, StripeGateway, RazorpayGateway — none of them change. This is SOLID in practice.

Q7. Calculate salary based on grades — Strategy pattern

Salary calculation via Strategy:

```
// Employee model
public class Employee
{
```

```
    public string Name  { get; set; }
    public decimal Basic { get; set; }
    public string Grade  { get; set; }   // A, B, C
}

// Strategy interface
public interface ISalaryStrategy { decimal Calculate(Employee e); }

// Grade-specific algorithms
public class GradeAStrategy : ISalaryStrategy
{
    // Grade A: basic + 100% basic (HRA) + 50% basic (allowances)
    public decimal Calculate(Employee e) => e.Basic + e.Basic + (e.Basic * 0.5m);
}
public class GradeBStrategy : ISalaryStrategy
{
    // Grade B: basic + 50% basic (HRA) + 20% basic (allowances)
    public decimal Calculate(Employee e) => e.Basic + (e.Basic * 0.5m) + (e.Basic *
0.2m);
}
public class GradeCStrategy : ISalaryStrategy
{
    // Grade C: basic + 10% HRA only
    public decimal Calculate(Employee e) => e.Basic + (e.Basic * 0.1m);
}

// Payroll — uses a lookup, no if-else chains
public class Payroll
{
    private readonly Dictionary<string, ISalaryStrategy> _strategies = new()
    {
        ["A"] = new GradeAStrategy(),
        ["B"] = new GradeBStrategy(),
        ["C"] = new GradeCStrategy()
    };

    public decimal GetGrossSalary(Employee emp)
    {
        if (!_strategies.TryGetValue(emp.Grade, out var strategy))
            throw new ArgumentException($"Unknown grade: {emp.Grade}");
        return strategy.Calculate(emp);
    }
}

// Usage
var payroll = new Payroll();
var emp = new Employee { Name = "Vamshi", Basic = 50000, Grade = "A" };
Console.WriteLine($"{emp.Name}: ₹{payroll.GetGrossSalary(emp):N0}");  // ₹1,25,000
```

To add Grade D: write GradeDStrategy class + add one entry to the dictionary. Payroll.GetGrossSalary() does not change — that is the Open/Closed principle working alongside Strategy.

Q8. Explain D from SOLID with written example

Dependency Inversion — before and after:

```
// ── BEFORE (VIOLATION): high-level depends on low-level concrete ─
public class ReportGenerator_Bad
{
    // Hard-wired to SQL — cannot swap to file or API
    private SqlReportRepository _repo = new SqlReportRepository();
    private SmtpEmailSender     _email = new SmtpEmailSender();

    public void GenerateAndSend(int reportId, string recipient)
    {
        var data   = _repo.GetReportData(reportId);    // concrete
        var pdf    = CreatePdf(data);
```

```
        _email.Send(recipient, "Report", pdf);          // concrete
    }
}
// To unit test — must have real SQL and real SMTP. Impossible to isolate.

// ── AFTER (CORRECT): depend on abstractions ──────────────────────
public interface IReportRepository { ReportData Get(int id); }
public interface IEmailSender      { void Send(string to, string subject, byte[]
attachment); }

public class ReportGenerator_Good
{
    private readonly IReportRepository _repo;
    private readonly IEmailSender      _email;

    // Dependencies come IN via constructor — not created inside
    public ReportGenerator_Good(IReportRepository repo, IEmailSender email)
    {
        _repo  = repo;
        _email = email;
    }

    public void GenerateAndSend(int reportId, string recipient)
    {
        var data = _repo.Get(reportId);
        var pdf  = CreatePdf(data);
        _email.Send(recipient, "Report", pdf);
    }
}

// Production: real implementations
builder.Services.AddScoped<IReportRepository, SqlReportRepository>();
builder.Services.AddScoped<IEmailSender,      SmtpEmailSender>();

// Tests: fake implementations
var mockRepo  = new Mock<IReportRepository>();
var mockEmail = new Mock<IEmailSender>();
var generator = new ReportGenerator_Good(mockRepo.Object, mockEmail.Object);
// Test without SQL or SMTP
```

The "inversion" is that control of which concrete class to use moves from inside the class to outside (the DI container). This enables testing and flexibility.

Q9. Any design pattern you implemented — Singleton

Implemented AppLogger as Singleton using Lazy<T>. Single log writer avoids multiple file handles. Thread-safe. Configuration loaded once and shared across all requests.

Q10. How to pass data between controllers

TempData: survives one redirect (most common). Route values: pass in URL. Session: persist across requests. Shared service via DI: inject same service into both controllers.

Q11. AutoMapper

Library that maps one object to another by convention (matching property names). Eliminates manual property assignment. CreateMap<Employee, EmployeeDto>(); mapper.Map<EmployeeDto>(employee);

Q12. Bundles.config

ASP.NET MVC bundling configuration: combines multiple JS/CSS files into one download + minifies. Reduces HTTP requests. Not available in ASP.NET Core — use Webpack, Vite, or bundleconfig.json.

Q13. How to implement caching

In-memory: IMemoryCache with GetOrCreateAsync(key, factory, expiry). Distributed: IDistributedCache backed by Redis. Output: [ResponseCache(Duration=60)]. EF Core: second-level cache via extension.

Q14. Difference between list and dictionary

List<T>: ordered by index, O(n) search, duplicates allowed. Dictionary<K,V>: O(1) lookup by key, keys must be unique. Use List when order matters; Dictionary when fast lookup by key is needed.

Q15. How to implement security

HTTPS everywhere, JWT/OAuth2 authentication, role/policy authorization, input validation (FluentValidation), parameterised queries (no string concat), CORS policy, rate limiting, secrets in Key Vault, [Authorize] on controllers.

Q16. JWT token

Three Base64URL parts joined by dots: Header ({"alg":"HS256","typ":"JWT"}) . Payload ({"sub":"42","role":"Admin","exp":1700000000}) . Signature (HMAC of header+payload with secret). Validated on every request by middleware.

Q17. Return type in API

IActionResult: most flexible (Ok/NotFound/BadRequest etc). ActionResult<T>: typed + flexible. Task<IActionResult>: async. Specific type: treated as 200 OK automatically.

Q18. How to handle exception on application level

Global ExceptionHandlingMiddleware catches all unhandled exceptions, logs them, returns consistent ProblemDetails JSON response. Exception filters for MVC-specific. Never expose stack traces in production.

SECTION 8: ANGULAR (135 Questions)

Q1. Why Angular?

Component-based SPA framework. TypeScript-first (type safety). Built-in DI, routing, HTTP client, reactive forms. Two-way binding reduces boilerplate. Strong CLI. Used by enterprise teams at scale.

Q2. Which is the latest version?

Angular 18 (May 2024). Key features: Signals (reactive primitives replacing RxJS for simple state), new control flow (@if, @for, @switch replacing *ngIf/*ngFor), standalone components default, deferred loading (@defer).

Q3. Difference JS and Angular?

JavaScript: scripting language — runs in browser. Angular: full application framework built WITH TypeScript — includes component system, routing, DI, HTTP, forms, CLI toolchain.

Q4. How to install Angular?

npm install -g @angular/cli → ng new my-app (choose routing: yes, styles: scss) → cd my-app → ng serve (dev server at localhost:4200)

Q5. What is AOT?

Ahead-of-Time compilation: Angular compiles HTML templates at BUILD time into efficient JavaScript. Smaller bundle, faster startup, template errors caught at build (not runtime). Enabled by default in ng build --prod.

Q6. What are decorators?

TypeScript metadata annotations. @Component tells Angular this class is a component. @Injectable marks it for DI. @Input/@Output define data flow. Angular reads decorators at runtime to wire everything up.

Q7. How do you create a component?

ng generate component user-list (shortcut: ng g c user-list). Creates user-list.component.ts, .html, .scss, .spec.ts. Registers in nearest NgModule (or standalone if --standalone flag used).

Q8. How does Angular app start?

main.ts → bootstrapApplication(AppComponent, appConfig) → Angular reads AppComponent @Component → creates component tree → Router loads first matched route → renders in <router-outlet>.

Q9. Package.json vs package.lock.json?

package.json: your declarations — "I need RxJS ^7.0". package.lock.json: exact installed versions — "RxJS 7.8.1 from this exact URL". Always commit lock file for reproducible builds.

Q10. How to download/install packages?

npm install rxjs (adds to dependencies). npm install --save-dev @types/node (dev dependency). npm install --legacy-peer-deps (if peer dep conflict).

Q11. What is angular.json file?

Workspace configuration file: where source files are, output path, assets, global styles, scripts, budget limits, build optimisations, test config. One entry per project in the workspace.

Q12. What are modules?

@NgModule: groups related components, directives, pipes. Declares what it has. Imports what it needs. Exports what others can use. In Angular 14+ standalone components can replace modules.

Q13. What is ngModel?

Two-way binding directive: [(ngModel)]="property". Input change updates property; property change updates input. Requires FormsModule import. Under the hood: [ngModel] + (ngModelChange).

Q14. What is routing and how is it implemented?

RouterModule.forRoot(routes) in AppModule (or provideRouter(routes) in standalone). Routes array: { path: "orders", component: OrdersComponent }. Template: <router-outlet>. Navigate: router.navigate(["/orders"]).

Q15. Lazy loading in routing?

loadComponent: () => import("./orders/orders.component").then(m => m.OrdersComponent)

Module lazy: loadChildren: () => import("./admin/admin.module").then(m => m.AdminModule)

Angular compiles each lazy chunk into a separate JS file — not downloaded until user navigates there.

Q16. What are directives and their types?

STRUCTURAL (change DOM): *ngIf, *ngFor, *ngSwitch — prefix * is syntactic sugar for <ng-template>.

ATTRIBUTE (change appearance/behaviour): ngClass, ngStyle, custom directives.
COMPONENT: a directive with a template (most used type).

Q17. How to create custom Directives?

@Directive({ selector: "[appHighlight]" }) class HighlightDirective { constructor(private el: ElementRef, private renderer: Renderer2) {} @HostListener("mouseenter") onEnter() { this.renderer.setStyle(this.el.nativeElement, "background", "yellow"); } }

Q18. Types of data binding?

ONE-WAY component→view: Interpolation {{ name }}, Property binding [src]="imageUrl".
ONE-WAY view→component: Event binding (click)="handleClick()".
TWO-WAY: [(ngModel)]="username" — shorthand for [ngModel]="x" (ngModelChange)="x=$event".

Q19. What is string interpolation?

{{ expression }} in template. Angular evaluates expression in component context and inserts result as TEXT into the DOM. Cannot call methods with side effects. Cannot use new, =, +=.

Q20. String interpolation vs Property binding?

Interpolation {{ val }}: outputs TEXT only — use in text content.
Property binding [property]="val": sets DOM PROPERTY — use for src, disabled, class, style etc.
[src]="imageUrl" is correct; {{ imageUrl }} in src attribute is old Angular 1 style.

Q21. What is HTTP Interceptor? How to implement it?

Global middleware for HttpClient calls. Intercepts every request/response before they reach handlers. Use for: add JWT header, log all calls, show spinner, refresh expired tokens, transform errors.

Q22. What are services in Angular?

@Injectable classes providing shared logic and data. Singleton at root level (providedIn: "root"). No component lifecycle dependency. Can be injected into any component or other service.

Q23. Dependency injection in Angular?

Angular DI container creates and provides service instances. Services declared in providers or with providedIn. Hierarchical: root injector → module injector → component injector. Child gets parent service unless it provides its own.

Q24. Ways to pass data between components?

Parent→Child: @Input(). Child→Parent: @Output() EventEmitter. Siblings: shared service with Subject/BehaviorSubject. Deep tree: NgRx store or React-style context. URL: Router params/query params.

Q25. All Life cycle hooks in sequence?

ngOnChanges (before init if @Input) → ngOnInit (once, after first change detection) → ngDoCheck (every CD cycle) → ngAfterContentInit (once after ng-content projected) → ngAfterContentChecked (every CD) → ngAfterViewInit (once after view+children init) → ngAfterViewChecked (every CD) → ngOnDestroy (before removal).

Q26. Constructor vs ngOnInit

Constructor: TypeScript class initialisation — should ONLY inject dependencies, nothing else. Angular DI calls it.

ngOnInit: Angular lifecycle hook — safe to access @Input values, call HTTP services, setup subscriptions. Called after constructor and first change detection.

Q27. ngOnChanges vs ngDoCheck

ngOnChanges: fires when @Input REFERENCE changes (new array/object reference). Has SimpleChanges argument showing old and new value. Shallow — does not detect array item changes.

ngDoCheck: fires on EVERY change detection cycle — lets you implement custom comparison. Expensive if heavy logic inside.

Q28. What is ViewChild?

@ViewChild("myRef") myEl: ElementRef — gets direct reference to a child DOM element or component. Available after ngAfterViewInit. @ViewChildren gives QueryList of multiple matches.

Q29. How to create API calls in Angular?

Inject HttpClient. this.http.get<User[]>("/api/users") returns Observable. Subscribe in component or use async pipe. Use pipe(catchError()) for error handling. Return from service, subscribe in component.

Q30. HTTP filters in Angular?

Angular has HTTP INTERCEPTORS, not filters. See Q21. Intercept requests/responses globally.

Q31. How to add auth header in HTTP call?

Option 1: Interceptor (recommended — automatic for every call). Option 2: httpOptions: const headers = new HttpHeaders({ Authorization: "Bearer " + token }); this.http.post(url, body, { headers });

Q32. What are Pipes and their types?

Transforms displayed values in template. {{ date | date:"dd/MM/yyyy" }}. Built-in: date, currency, uppercase, lowercase, percent, async, json, slice, decimal.

PURE (default): runs only when input reference changes.

IMPURE: runs on every change detection cycle — use carefully (slow).

Q33. How to create a Pipe?

@Pipe({ name: "rupee", pure: true }) export class RupeePipe implements PipeTransform { transform(value: number): string { return "₹" + value.toLocaleString("en-IN"); } }

Usage: {{ salary | rupee }}

Q34. Promise vs Observables

Promise: single value, starts immediately (eager), cannot cancel, .then/.catch.

Observable: multiple values over time, starts only when subscribed (lazy), cancellable with unsubscribe(), rich operators (map, filter, retry, debounce), works with async pipe.

Q35. Observables vs Subject

Observable: COLD — each subscriber gets its own execution (separate HTTP call per subscriber).

Subject: HOT — all subscribers share one stream. Subject is both Observable AND Observer — you can call .next() on it externally.

Q36. Different Types of Subject

Subject: no initial value, late subscribers miss past emissions.

BehaviorSubject(initialValue): holds LAST value, new subscriber gets it immediately. Most used.
ReplaySubject(n): replays last N values to new subscribers.
AsyncSubject: emits ONLY the last value, ONLY on complete.

Q37. Async programming in Angular?

Yes: Observables (HttpClient, RxJS), Promises (async/await), async pipe in templates. Angular change detection handles async automatically when using async pipe or zone.js.

Q38. Implement asynchronous programming in Angular?

HTTP: inject HttpClient, return observable from service, subscribe in component OR use async pipe. Multiple parallel: forkJoin([call1, call2]). Sequential: switchMap. Error handling: catchError().

Q39. Template driven vs Reactive forms?

Template driven: ngModel in template, simple, less code, FormsModule, hard to unit test.
Reactive: FormGroup/FormControl in component TypeScript, explicit, unit testable, dynamic validation, ReactiveFormsModule. PREFER for any form with validation logic.

Q40. What is auth guard in Angular?

Implements CanActivate interface. Returns true (allow navigation) or false/UrlTree (redirect to login). Protects routes from unauthenticated users. Runs BEFORE the component is created.

Q41. Types of auth guards?

CanActivate (protect route), CanActivateChild (protect child routes), CanDeactivate (prevent leaving with unsaved data), CanLoad (prevent loading lazy module), CanMatch (Angular 15+, functional guards preferred).

Q42. What is local storage?

Browser key-value store. Max ~5MB. Persists after browser close. Same origin only. localStorage.setItem("token", jwt); localStorage.getItem("token"); localStorage.removeItem("token"). Do NOT store sensitive data without encryption.

Q43. Have you used RxJS? Why?

Yes. RxJS makes async operations composable and readable. HTTP calls, user input debouncing, real-time data streams — all expressed as observable pipelines. Operators transform data without nested callbacks.

Q44. RxJS operators?

Creation: of(), from(), interval(), fromEvent().
Transform: map(), switchMap(), mergeMap(), concatMap().
Filter: filter(), debounceTime(), distinctUntilChanged(), take().
Combine: forkJoin(), combineLatest(), merge().
Utility: tap(), catchError(), retry(), finalize(), takeUntil().

Q45. What is map operator?

Transforms each emitted value. Like Array.map but for streams. search$.pipe(map(term => term.toUpperCase())) — every emission is uppercased before passing downstream.

Q46. Template Reference Variable?

#myRef on a template element. <input #nameInput> gives you reference to the input element. Access in TS via @ViewChild("nameInput") nameEl: ElementRef. Use in template: <button (click)="fn(nameInput.value)">Send</button>

Q47. Testing in Angular?

Jasmine + Karma (default). TestBed creates Angular testing module. Mock services with spyOn or jasmine.createSpyObj. ComponentFixture for component testing. Async tests with fakeAsync/tick or waitForAsync.

Q48. State management in Angular?

Local component state: property / BehaviorSubject in service.
Global state: NgRx (Redux pattern — Actions, Reducers, Store, Selectors, Effects).
Simple shared state: shared service with BehaviorSubject.
Signals (Angular 17+): built-in reactive primitive — simplest option for new code.

Q49. Angular new features (v17-18)?

@if, @for, @switch control flow (no more *ngIf/*ngFor directives). Signals for reactive state. Standalone components default. @defer blocks for deferred loading. New project structure. Built-in SSR setup.

Q50. NG serve vs NG build?

ng serve: in-memory dev server, no dist folder, HMR (hot reload), no optimisations — use during development.
ng build: creates physical dist/ folder, AOT, tree-shaking, minification, chunk splitting — use for deployment.

Angular Key Code Examples

QA1. HTTP Interceptor — JWT auth + error handling + spinner

auth.interceptor.ts:

```
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
    constructor(private auth: AuthService, private spinner: SpinnerService) {}

    intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
        const token = this.auth.getToken();

        // Clone request and add Authorization header (originals are immutable)
        const authReq = token
            ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
            : req;

        this.spinner.show();  // show loading indicator

        return next.handle(authReq).pipe(
            catchError((error: HttpErrorResponse) => {
                if (error.status === 401) {
                    // Token expired — redirect to login
                    this.auth.logout();
                }
                if (error.status === 403) {
                    // Not authorized — show error
                    console.error('Access denied');
                }
                return throwError(() => error);   // rethrow for component to handle
            }),
            finalize(() => this.spinner.hide())   // always hide spinner, even on error
        );
```

```
    }
}

// Register in app.config.ts or AppModule providers:
// { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true }
// multi:true = do not replace existing interceptors, ADD to the chain
```

Every HTTP request automatically gets the JWT header — no need to add it in every service. This is cross-cutting concerns handled in one place.

QA2. Reactive Form with validation + custom validator

registration.component.ts:

```
export class RegistrationComponent implements OnInit {
    form!: FormGroup;

    constructor(private fb: FormBuilder, private userSvc: UserService) {}

    ngOnInit() {
        this.form = this.fb.group({
            // [initial value, validators array, async validators array]
            name:     ['', [Validators.required, Validators.minLength(3)]],
            email:    ['', [Validators.required, Validators.email]],
            password: ['', [Validators.required, Validators.minLength(8)]],
            confirm:  ['', Validators.required],
            age:      [null, [Validators.required, Validators.min(18),
Validators.max(100)]]
        }, {
            // Cross-field validator: password must match confirm
            validators: this.passwordMatch
        });
    }

    // Custom cross-field validator — compares two controls
    passwordMatch(group: AbstractControl): ValidationErrors | null {
        const pw  = group.get('password')?.value;
        const cfm = group.get('confirm')?.value;
        return pw === cfm ? null : { mismatch: true };
    }

    // Convenience getter — shorter template access
    get f() { return this.form.controls; }

    onSubmit() {
        if (this.form.invalid) { this.form.markAllAsTouched(); return; }
        this.userSvc.register(this.form.value).subscribe({
            next: () => console.log('Registered!'),
            error: err => console.error(err)
        });
    }
}

// Template snippet
// <input formControlName="name">
// <div *ngIf="f['name'].errors?.['required'] && f['name'].touched">Name is
required</div>
// <div *ngIf="f['name'].errors?.['minlength']">Min 3 characters</div>
// <div *ngIf="form.errors?.['mismatch']">Passwords do not match</div>
// <button [disabled]="form.invalid">Register</button>
```

markAllAsTouched() forces validation messages to show even if user never touched the field — useful when user clicks Submit without filling form.

QA3. BehaviorSubject shared service + async pipe

cart.service.ts + header.component.ts:

```
// cart.service.ts
@Injectable({ providedIn: 'root' })
export class CartService {
    // BehaviorSubject: holds current count, emits to all subscribers
    private _count = new BehaviorSubject<number>(0);

    // Expose as read-only Observable so external code cannot call .next()
    count$ = this._count.asObservable();

    addItem(product: Product) {
        const current = this._count.value;   // synchronous access to current value
        this._count.next(current + 1);        // emit new value to all subscribers
    }

    removeItem() {
        this._count.next(Math.max(0, this._count.value - 1));
    }

    clearCart() { this._count.next(0); }
}

// header.component.ts — subscribes via async pipe (no manual unsubscribe needed)
@Component({
    selector: 'app-header',
    template: `
        <nav>
            <!-- async pipe: subscribes, gets latest value, auto-unsubscribes on destroy
-->
            <span class="badge">{{ cartCount$ | async }}</span>
        </nav>
    `
})
export class HeaderComponent {
    // Just declare the observable — no subscribe() needed in component!
    cartCount$ = this.cart.count$;
    constructor(private cart: CartService) {}
}

// product.component.ts
@Component({ template: '<button (click)="add()">Add to Cart</button>' })
export class ProductComponent {
    constructor(private cart: CartService) {}
    add() { this.cart.addItem(this.product); }
}
```

async pipe is better than subscribe() in component because it automatically unsubscribes when component is destroyed — no memory leaks. BehaviorSubject is the best choice for "current state" that new subscribers need immediately.

QA4. switchMap vs mergeMap — real scenario

search with switchMap, parallel calls with mergeMap:

```
// ── switchMap: CANCEL previous, use only latest ──────────────────
// Use case: search box — user types fast, only care about last search result
this.searchControl.valueChanges.pipe(
    debounceTime(300),          // wait 300ms after user stops typing
    distinctUntilChanged(),     // do not search if same value as before
    switchMap(term =>           // switchMap CANCELS previous HTTP call if new term
arrives
        this.productSvc.search(term).pipe(
            catchError(() => of([]))  // on error, return empty array (do not break
stream)
        )
    )
```

```
).subscribe(results => this.results = results);
// If user types "an", "ang", "angu" in quick succession:
// Only the "angu" HTTP call completes — earlier ones are cancelled. One network request.

// ── mergeMap: run ALL concurrently, merge results ──────────────────
// Use case: download multiple reports simultaneously
const reportIds = [1, 2, 3, 4, 5];
from(reportIds).pipe(
    mergeMap(id =>
        this.reportSvc.download(id)  // all 5 downloads start simultaneously
    )
).subscribe(report => this.addReport(report));
// All 5 HTTP calls fire at once — responses arrive as each completes (may be out of
order)

// ── concatMap: run SEQUENTIALLY, one by one ────────────────────────
// Use case: process queue — must wait for each to finish before next
from(orders).pipe(
    concatMap(order => this.orderSvc.process(order))  // processes order 1, waits, then
2, waits, then 3
).subscribe();
```

switchMap = latest wins (search, typeahead). mergeMap = all concurrent (parallel downloads). concatMap = one at a time in order (sequential processing). exhaustMap = ignore new until current completes (login button).

Q51. Latest Angular version

Angular 18 (May 2024). Key: Signals stable, new control flow stable, partial hydration for SSR, standalone default.

Q52. Directives and types

Structural (*ngIf,*ngFor), Attribute (ngClass,ngStyle,custom), Component (directive + template).

Q53. Decorator

TypeScript metadata: @Component, @NgModule, @Injectable, @Input, @Output, @Pipe, @Directive, @ViewChild.

Q54. Components

Building block: @Component({ selector, templateUrl, styleUrls }) class with TypeScript logic + lifecycle hooks.

Q55. Modules

@NgModule: groups declarations (components/directives/pipes), imports, exports, providers. Standalone components reduce need.

Q56. Template

HTML with Angular syntax: {{ }}, [ ], ( ), [( )], *ngIf, @if, structural directives, pipes.

Q57. Services

@Injectable shared logic classes: HTTP calls, state, utilities. Injected via constructor. Singleton at root.

Q58. Dependency Injection

Angular DI system manages service creation and sharing. Hierarchical injector tree. Constructor-based injection.

Q59. Async Pipe

Subscribes to Observable/Promise in template. Auto-unsubscribes on component destroy. {{ data$ | async }}.

Q60. Angular pipe

Transform display values in template. Pure (default) or impure. date, currency, async, custom.

Q61. @Output

EventEmitter in child. Parent listens: (myEvent)="handler($event)". Used for child→parent communication.

Q62. @Input

Property in child receives value from parent: [title]="parentValue". Triggers ngOnChanges on change.

Q63. ViewChild and ViewChildren

@ViewChild: single reference (element, component, directive). @ViewChildren: QueryList of multiple matches. Available in ngAfterViewInit.

Q64. Angular forms

Template-driven (FormsModule, ngModel) for simple. Reactive (ReactiveFormsModule, FormGroup) for complex with validation.

Q65. Routing

RouterModule.forRoot(routes). Route guards. Lazy loading. Named outlets. ActivatedRoute for params.

Q66. Lazy loading

Load feature module/component only when route accessed. Reduces initial bundle. loadComponent() or loadChildren().

Q67. DataBindings

Interpolation, property, event, two-way. All compile to efficient change detection checks.

Q68. Observable, Observer

Observable: sequence of values over time. Observer: { next(val), error(err), complete() } callbacks. subscribe() connects them.

Q69. Purpose of RxJS

Reactive programming library. Makes async code composable, readable, cancellable. Operators for transform, filter, combine streams.

Q70. Different RxJS operators

map, filter, switchMap, mergeMap, concatMap, debounceTime, distinctUntilChanged, forkJoin, combineLatest, takeUntil, catchError, retry, tap, share.

Q71. Difference RxJS and Promise

Promise: one value, eager, non-cancellable. Observable: multiple values, lazy, cancellable, operators. HttpClient returns Observable for this reason.

Q72. Angular task in recent project

Prepare: components built, lazy routes configured, interceptors for JWT, reactive forms with validation, custom pipes, RxJS operators used, state management approach.

Q73. Angular 10 vs 11

v11: strict mode enabled by default, webpack 5, HMR default, updated Angular Material. v10: strict templates optional, TypeScript 4.0, new date range picker.

Q74. Directives in Angular and types

See Q16.

Q75. Encapsulation in Angular

ViewEncapsulation.Emulated (default): scoped CSS via attribute selectors. None: styles go global. ShadowDom: native browser shadow DOM isolation.

Q76. Angular controllers

AngularJS (v1) concept. Angular 2+ uses COMPONENTS instead of controllers + views.

Q77. ngOnInit vs ngOnChanges

ngOnInit: once, after first change detection. ngOnChanges: every time @Input reference changes (including before ngOnInit on first render).

Q78. String interpolation in Angular

{{ expression }} — evaluates in component context, converts to string, renders as text content.

Q79. Retrieve data in one tab in another new tab without API call

localStorage (shared across tabs, same origin). BroadcastChannel API (real-time messaging between tabs). SharedWorker.

Q80. Explain lazy loading — how achieved

loadComponent/loadChildren in routes. Route split into separate chunk at build time. Chrome DevTools Network tab shows chunk loaded on navigation.

Q81. @Input @Output — parent communication

Parent passes data to child via @Input. Child notifies parent via @Output EventEmitter. Parent handles with (event)="handler($event)".

Q82. Observable vs Promises

See Q34.

Q83. Different types of directives

See Q16.

Q84. Angular life cycle hooks

See Q25.

Q85. Local storage vs session storage

localStorage: persists forever (until cleared). sessionStorage: only current TAB/SESSION — cleared on tab close.

Q86. Angular localization

@angular/localize package. i18n attributes in template. ng extract-i18n generates messages.xlf. Build per locale: ng build --localize.

Q87. Sessions in Angular

Angular is client-side — no server sessions natively. Use localStorage/sessionStorage for client state. Auth handled via JWT in localStorage/cookie.

Q88. RxJS operators

See Q44.

Q89. switchMap and mergeMap

See code example A4 above.

Q90. Server-side rendering in Angular

Angular Universal (now built-in Angular 17+). Pre-renders HTML on server for faster first paint and SEO. ng add @angular/ssr adds it.

Q91. Ways to pass data between components

@Input/@Output (parent-child), shared service BehaviorSubject (siblings/distant), NgRx store (global), Router params.

Q92. Pipes

See Q32-33.

Q93. AOT compiler

Compiles at build time. Catches template errors early. Smaller bundle (no compiler shipped). Production default.

Q94. Angular elements

Package Angular components as standard Web Components (CustomElements). Embed in non-Angular apps (React, Vue, plain HTML).

Q95. Interceptors

See Q21 + code example A1.

Q96. How state management works (NgRx)

Dispatch Action → Reducer computes new state → Store updates → Selector emits new slice → Component re-renders. Effects handle side effects (HTTP) asynchronously.

Q97. Session storage vs Local storage

See Q85.

Q98. mergeMap explain

Projects each source value to inner Observable, subscribes to ALL concurrently, merges all outputs. No cancellation — all run. See A4 example.

Q99. Share data between 2 tabs

BroadcastChannel API: new BroadcastChannel("cart"); channel.postMessage(data); channel.onmessage = e => update(e.data). Or localStorage storage event.

Q100. Share data between 2 components

Shared service with BehaviorSubject. Both components inject same service. Service holds state, components subscribe.

Q101. Cookies and how created

document.cookie = "key=value; expires=...;path=/;SameSite=Strict". HttpOnly cookies set by server (cannot read from JS — more secure for auth tokens).

Q102. Observable vs Promises difference

See Q34.

Q103. Difficulty migrating Angular versions

Run ng update @angular/core @angular/cli. Read migration guide. Common issues: ivy renderer changes, RxJS operator renames, ViewChild static flag, stricter templates.

Q104. dependencies vs devDependencies

dependencies: shipped to production (Angular, RxJS). devDependencies: build/test only (@angular/cli, TypeScript, Karma, Jasmine). ng build includes only dependencies.

Q105. Where angular.json initialized

Project root, created by ng new. Edit manually or via ng config command. CI/CD reads it for build commands.

Q106. localStorage capacity

~5MB per origin (browser enforced). Cannot be increased programmatically. Use IndexedDB for larger client-side storage.

Q107. module imports, exports, providers

imports: other modules this module needs. declarations: components/directives/pipes this module owns. exports: subset of declarations available to other modules. providers: services (usually use providedIn:root instead).

Q108. actions and reducers (NgRx)

Action: { type: "[Cart] Add Item", payload: product }. Reducer: pure function (state, action) => newState — no async, no side effects, returns new state object.

Q109. Where pipes initialized

Declared in the NgModule that declares the component using them. Or imported standalone via imports array.

Q110. main.ts purpose

Application entry point. Calls bootstrapApplication(AppComponent, appConfig) which starts Angular. Imported in index.html as a script module.

Q111. angular 13 vs angular 8

v13: Ivy only (no View Engine), persistent cache (faster builds), no IE11, ESBuild. v8: Ivy optional preview, Bazel experimental, DI improvements.

Q112. Save error state in NgRx — refresh?

Yes: interface State { data: Product[]; error: string | null; loading: boolean; }. Reducer sets error on failure. If user refreshes, dispatch load action in ngOnInit — component re-fetches.

Q113. Pipe vs directive

Pipe: transform VALUE in template (display only). Directive: change DOM BEHAVIOUR (add CSS, handle events, add/remove elements). Different jobs, different APIs.

Q114. AGgrid

Third-party high-performance data grid. Virtual scrolling (renders only visible rows). Features: sort, filter, group, edit, export. Works with Angular via @ag-grid-community/angular.

Q115. What are Reactive forms

FormGroup holds FormControls. Defined in TypeScript. Explicit validation. Reactive updates. Test without rendering. ReactiveFormsModule needed.

Q116. What is BehaviorSubject

RxJS Subject with initial value. Late subscribers always get latest emission. Expose as Observable for read-only access. Perfect for "current state" pattern.

Q117. AOT and JIT

AOT: compile at build time (production). JIT: compile at runtime in browser (old dev mode). AOT is faster and smaller — always use in production.

Q118. Increase localStorage capacity

No — cannot exceed ~5MB browser limit. Use IndexedDB (unlimited in practice) or server-side storage.

Q119. angular.json for?

Workspace config: source paths, output paths, assets, global styles, scripts, build optimisations (sourceMap, budgets), test config.

Q120. Synchronous vs asynchronous

Sync: blocks until done (reading file sync). Async: starts operation, continues, gets notified when done (HTTP call). JavaScript is single-threaded — async allows non-blocking I/O.

Q121. Data binding in Angular and types

See Q18.

Q122. Angular 11 features

Stricter template type checking. HMR enabled by default. Webpack 5. Font inlining. Updated CLI and CDK.

Q123. Where configure angular.json

Edited directly in the root angular.json file. Or use ng config project.architect.build.options.outputPath dist/myapp.

Q124. Change detection strategy

Default: check ALL components on every browser event. OnPush: check component only when @Input reference changes OR async pipe emits OR markForCheck() called. OnPush = big performance win for large apps.

Q125. Default vs OnPush change detection

Default: any click/xhr/timer → Angular checks entire component tree.

OnPush: Angular skips component unless its Input object reference changed. Pairs with immutable data and Observable/async pipe.

Q126. @Input @Output data transfer

Parent: <app-child [data]="myData" (dataChange)="onDataChange($event)">

Child: @Input() data; @Output() dataChange = new EventEmitter();

Child calls: this.dataChange.emit(newValue);

Q127. CSS pre-processors

Sass/SCSS, Less, Stylus. Extend CSS with variables ($color: red), nesting, mixins (@mixin flex-center), functions, inheritance. Compiled to plain CSS at build time. Angular supports SCSS natively.

Q128. Sass vs CSS

CSS: standard, no variables/nesting. SCSS: superset of CSS, adds $variables, nesting rules, @mixin, @extend, @import, @each loops. Compiled to CSS — browsers never see SCSS.

Q129. Center div inside div

Flexbox: .parent { display:flex; justify-content:center; align-items:center; height:100%; }

Grid: .parent { display:grid; place-items:center; }

Absolute: .child { position:absolute; top:50%; left:50%; transform:translate(-50%,-50%); }

Q130. Remove duplicates from array

JS: [...new Set(array)] or array.filter((v,i,a) => a.indexOf(v) === i)

Angular: in pipe transform or via LINQ equivalent in TypeScript.

Q131. data binding

See Q18 — interpolation, property, event, two-way binding.

Q132. content projection (ng-content)

Pass HTML from parent into child slot. <ng-content> in child template. Named slots: <ng-content select=".header">. Like React children prop.

Q133. calculator live coding

See Angular component below in code examples.

Q134. inlineblock vs block

block: full width, starts new line (div, p, h1). inline: only content wide, same line (span, a). inline-block: same line BUT can set width/height (button, img, input by default).

Q135. CSS grid vs flexbox

Flexbox: ONE dimension (row OR column). Best for component-level alignment. Grid: TWO dimensions (rows AND columns simultaneously). Best for page layout. Use both together: grid for layout, flex for alignment within cells.